

Lecture 2: Image Classification

Waitlisted Students

- We'll continue sending overrides in waitlist order as seats open
- If you get one, **enroll right now** – override will expire in 24h
- If you received an override and it expires, **you will not get another**

Office Hours

Office hours start this week; check Google Calendar for details

Fill out the following form about your office hour time preferences:

<https://forms.gle/Qh41oXuzjrTxFEVk9>

Piazza

Reminder: Piazza is our main source of communication this semester

Right now (enrolled students) < (enrolled students on Piazza)

Go enroll on Piazza if you haven't already

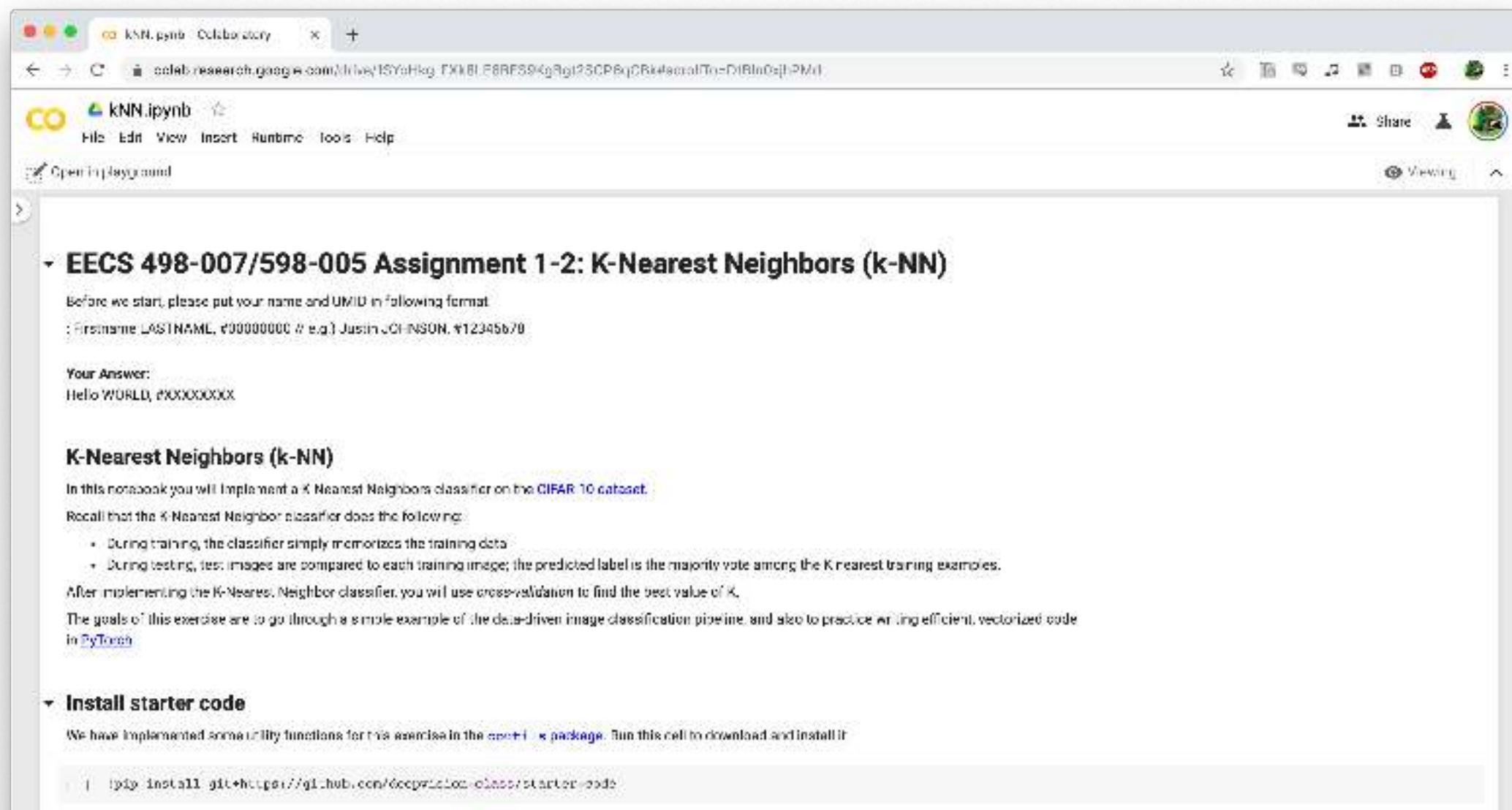
Assignment 1 Released

- <https://web.eecs.umich.edu/~justincj/teaching/eecs498/WI2022/assignment1.html>
- Uses Python, PyTorch, and Google Colab
- Introduction to PyTorch Tensors
- K-Nearest Neighbor classification
- Two *challenge questions* worth 2% each
- Due **Friday January 14, 11:59pm ET**

Assignment 1 Released

- <https://web.eecs.umich.edu/~justincj/teaching/eecs498/WI2022/assignment1.html>
- Make sure you download the **WI2022** version of the assignment!
- We released a slightly updated assignment today with minor bugfixes; see Piazza: <https://piazza.com/class/kxtai72amx34p0?cid=46>
- Only **enrolled students** can submit the assignment, but if you enroll late we will give you extra days to submit A1

Google Colab: Cloud Computing in the Browser



Google Colab: Cloud Computing in the Browser

EECS 498-007 / 598-005
Deep Learning for Computer Vision
Fall 2020

Colab Tutorial

What is Colab?

- Colaboratory is a Google research project created to help disseminate machine learning education and research. It's a Jupyter notebook environment that requires no setup to use and runs entirely in the cloud. (from [Google Colab Notebooks page](#))
- It allows you to use virtual machines with a GPU (or TPU) to accelerate machinelearning workloads for up to 12 hours at a time.
- It is **free to use!** There is a paid option called [Colab Pro](#) which gives access to faster GPUs, more RAM, more CPU cores, more disk space, and longer runtimes, those won't be necessary for this course.

Steps to use Colab

We've written a Colab tutorial:
[https://web.eecs.umich.edu/~j
ustincj/teaching/eecs498/WI2
022/colab.html](https://web.eecs.umich.edu/~justincj/teaching/eecs498/WI2022/colab.html)

Lecture 2: Image Classification

Image Classification: A core computer vision task

Input: image



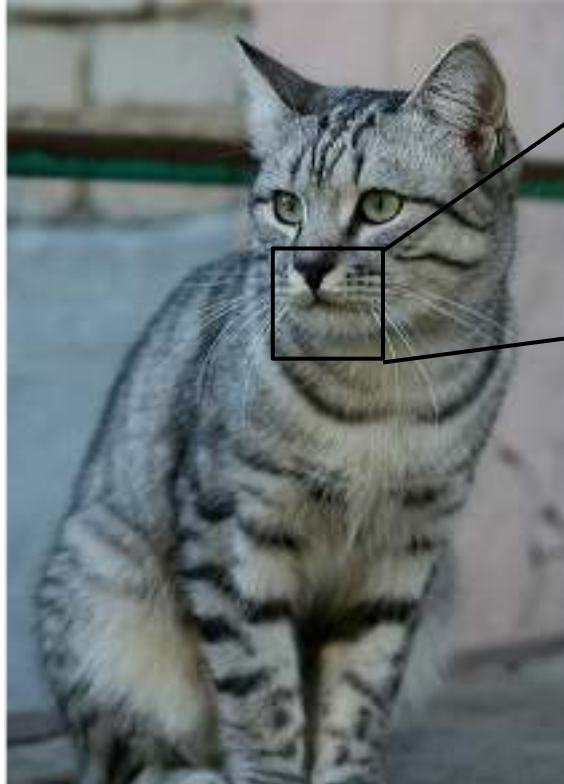
[This image by Nikita](#) is
licensed under [CC-BY 2.0](#)

Output: Assign image to one
of a fixed set of categories



cat
bird
deer
dog
truck

Problem: Semantic Gap



This image by Nikita is
licensed under [CC-BY 2.0](#)

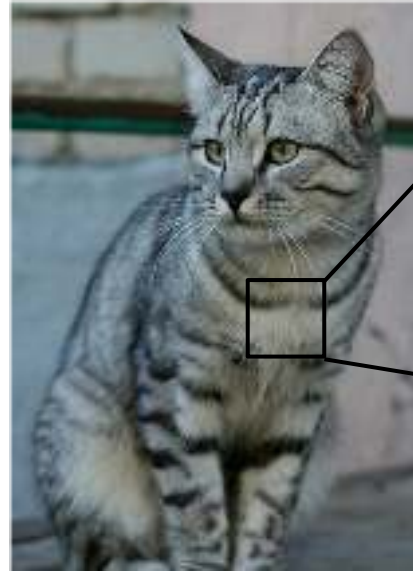
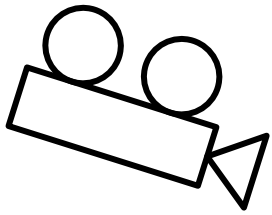
[105	112	108	111	104	99	106	99	95	103	112	119	104	97	93	87]
[91	98	102	106	104	79	90	103	99	105	123	136	110	105	94	85]
[76	85	90	105	128	105	87	96	95	99	115	112	100	103	99	85]
[99	81	81	93	128	131	127	108	95	98	102	99	98	93	101	94]
[105	91	61	64	60	91	88	85	101	107	109	98	75	84	96	95]
[114	108	85	55	55	60	64	54	64	87	112	120	98	74	84	91]
[133	137	147	103	65	81	88	65	52	54	74	84	102	93	85	82]
[128	137	144	140	109	95	86	70	62	65	63	63	60	73	86	101]
[125	133	140	137	119	121	117	94	65	79	80	65	54	64	72	98]
[127	125	131	147	133	127	126	131	111	96	89	75	61	64	72	84]
[115	114	104	123	150	148	131	118	113	109	100	92	74	65	72	78]
[89	93	90	97	188	147	131	118	113	114	113	100	108	95	77	88]
[63	77	86	81	77	70	102	123	117	115	117	125	125	130	115	87]
[62	65	82	89	78	71	88	101	124	126	119	101	107	114	131	119]
[63	65	75	88	89	71	62	81	120	138	135	105	81	98	118	118]
[87	65	71	87	100	95	69	45	76	130	126	107	92	94	105	112]
[118	97	82	86	117	123	116	66	41	51	95	93	89	95	102	107]
[164	146	112	88	82	178	124	104	76	48	45	66	88	101	102	109]
[157	178	147	128	98	86	114	132	112	97	69	55	78	82	99	94]
[130	128	134	161	139	100	109	118	121	134	114	87	65	53	69	86]
[128	112	96	117	150	144	120	115	104	107	102	93	87	81	72	79]
[123	107	96	86	83	112	153	149	122	109	104	75	80	107	112	99]
[122	121	102	88	82	86	94	117	145	148	153	102	58	78	92	107]
[122	164	148	103	71	56	78	83	93	103	119	139	102	61	69	84]

What the computer sees

An image is just a big grid of numbers between [0, 255]:

e.g. 800 x 600 x 3
(3 channels RGB)

Challenges: Viewpoint Variation



1120	113	106	111	104	58	105	63	55	163	112	119	104	97	90	171
91	100	140	140	144	14	58	101	93	104	124	144	114	134	91	493
176	95	94	100	130	105	67	55	55	59	115	112	185	185	99	303
80	101	101	103	100	115	127	180	84	88	102	84	84	84	84	101
1046	91	61	64	65	51	68	65	161	167	183	55	75	34	96	263
1117	106	105	105	105	68	64	54	61	87	112	124	84	27	87	493
1135	157	147	167	68	61	68	65	55	84	74	84	187	55	55	513
1120	127	174	170	166	65	64	10	62	64	64	64	64	20	86	1403
1154	155	148	157	115	117	117	54	65	75	85	85	84	64	77	363
1124	145	145	147	144	143	143	143	143	84	84	74	64	64	22	493
1114	114	100	109	146	148	171	118	119	183	183	10	74	84	77	753
180	92	96	97	106	147	112	118	112	119	112	184	184	64	27	493
93	77	108	103	117	14	163	118	117	118	117	124	124	104	114	613
102	55	55	55	76	71	56	162	124	124	124	181	187	144	101	1457
93	84	119	108	108	117	87	87	123	114	114	114	81	93	114	1183
137	55	71	57	106	55	68	45	70	129	125	187	92	24	185	1127
1110	91	10	108	117	113	118	88	47	61	64	84	84	84	100	1013
1134	146	113	94	52	126	124	164	75	49	45	55	55	181	192	1903
1147	149	141	118	88	88	114	112	112	87	84	64	74	67	89	443
1136	126	134	161	126	166	165	118	121	124	114	87	55	55	59	563
1130	112	108	117	118	144	128	114	181	187	183	84	87	81	22	263
1125	147	94	64	63	112	115	149	122	183	184	75	89	197	117	363
1124	147	160	135	139	84	117	144	144	144	144	144	144	144	144	144
1125	144	144	144	144	144	144	144	144	144	144	144	144	144	144	144

All pixels change when the camera moves!

Challenges: Intraclass Variation



[This image is CC0 1.0 public domain](#)

Challenges: Fine-Grained Categories

Maine Coon



[This image](#) is free for for use under the [Pixabay License](#)

Ragdoll



[This image](#) is [CC0 public domain](#)

American Shorthair



[This image](#) is [CC0 public domain](#)

Challenges: Background Clutter



[This image](#) is [CC0 1.0](#) public domain



[This image](#) is [CC0 1.0](#) public domain

Challenges: Illumination Changes



[This image is CC0 1.0 public domain](#)



[This image is CC0 1.0 public domain](#)



[This image is CC0 1.0 public domain](#)



[This image is CC0 1.0 public domain](#)

Challenges: Deformation



This image by Umberto Salvagnin is licensed under CC-BY 2.0



This image by Umberto Salvagnin is licensed under CC-BY 2.0



This image by sare bear is licensed under CC-BY 2.0



This image by Tom Thai is licensed under CC-BY 2.0

Challenges: Occlusion



This image is [CC0 1.0](#) public domain



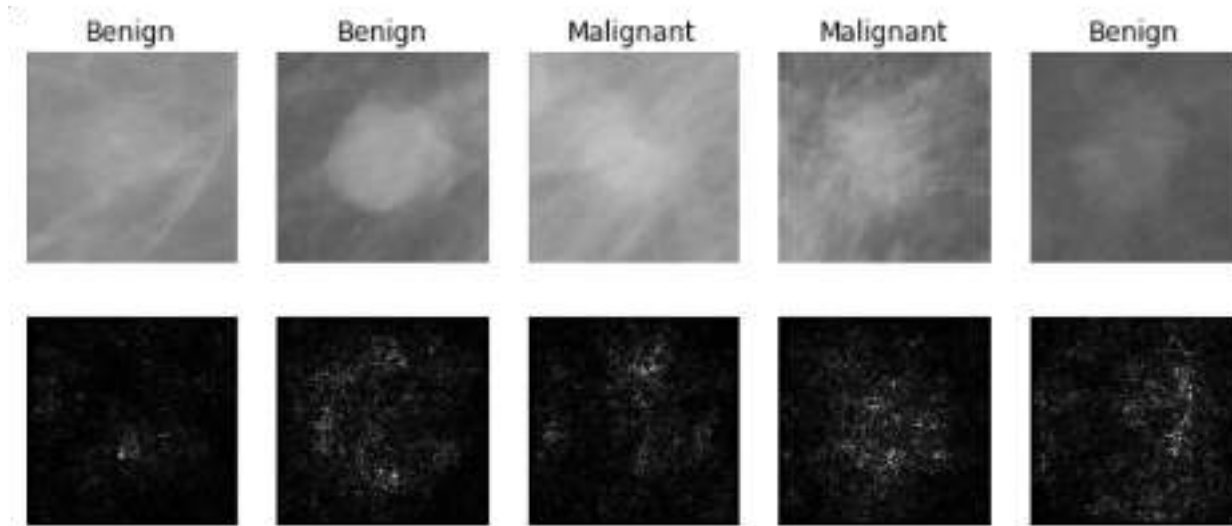
This image is [CC0 1.0](#) public domain



This image by [jonsson](#) is licensed under [CC-BY 2.0](#)

Image Classification: Very Useful!

Medical Imaging



Levy et al, 2016 Figure reproduced with permission

Galaxy Classification



Dieleman et al, 2014

From left to right: [public domain by NASA](#), usage permitted by ESA/Hubble, [public domain by NASA](#), and [public domain](#).

Whale recognition

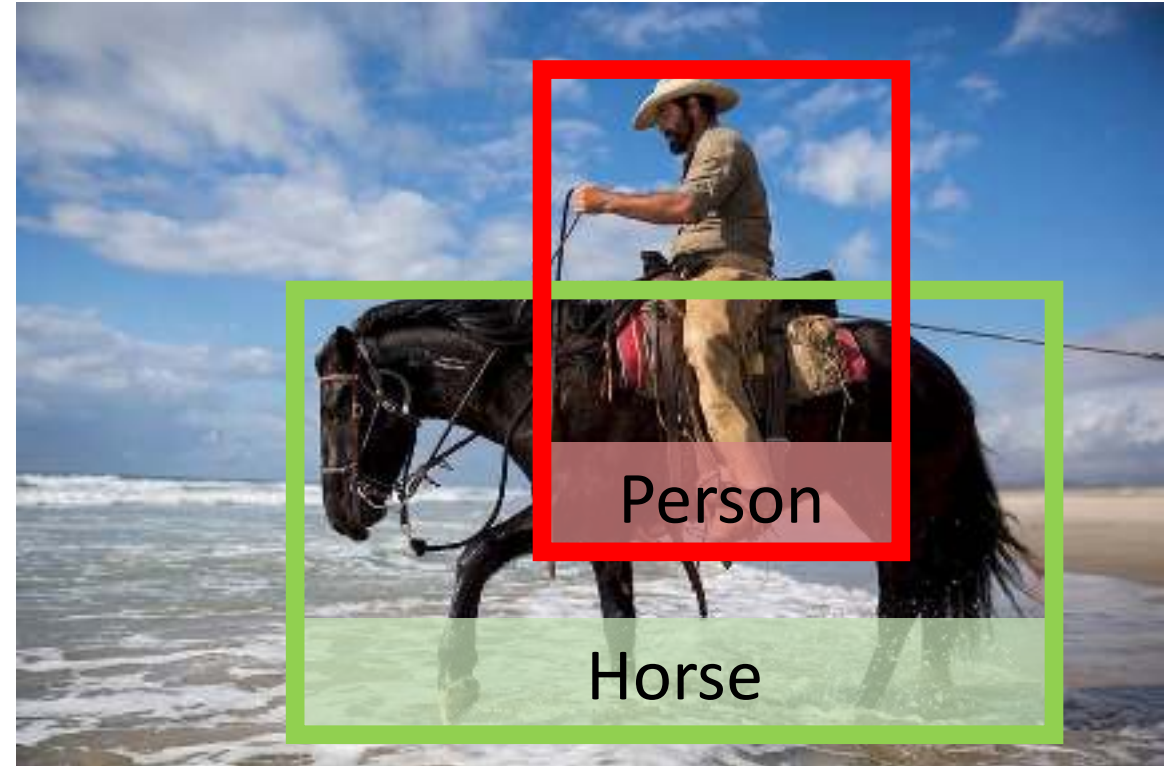


[Kaggle Challenge](#)

This image by Christin Khan is in the public domain and originally came from the U.S. NOAA.

Image Classification: Building Block for other tasks!

Example: Object Detection



[This image](#) is free to use under the [Pexels license](#)

Image Classification: Building Block for other tasks!

Example: Object Detection



[This image](#) is free to use under the [Pexels license](#)



Background

Horse

Person

Car

Truck

Image Classification: Building Block for other tasks!

Example: Object Detection



[This image](#) is free to use under the [Pexels license](#)



Background
Horse
Person
Car
Truck

Image Classification: Building Block for other tasks!

Example: Image Captioning



[This image](#) is free to use under the [Pexels license](#)



riding
cat
horse
man
when
...
<STOP>

What word
to say next?

Caption:
Man

Image Classification: Building Block for other tasks!

Example: Image Captioning



[This image](#) is free to use under the [Pexels license](#)



riding

cat

horse

man

when

...

<STOP>

What word
to say next?

Caption:
Man riding

Image Classification: Building Block for other tasks!

Example: Image Captioning



[This image](#) is free to use under the [Pexels license](#)



riding

cat

horse

man

when

...

<STOP>

What word
to say next?

Caption:
Man riding horse

Image Classification: Building Block for other tasks!

Example: Image Captioning



[This image](#) is free to use under the [Pexels license](#)



riding
cat
horse
man
when
...
<STOP>

What word
to say next?

Caption:
Man riding horse

Image Classification: Building Block for other tasks!

Example: Playing Go



[This image is CC0 public domain](#)



(1, 1)

(1, 2)

...

(1, 19)

...

(19, 19)

Where to
play next?

An Image Classifier

```
def classify_image(image):  
    # Some magic here?  
    return class_label
```

Unlike e.g. sorting a list of numbers,

no obvious way to hard-code the algorithm
for recognizing a cat, or other classes.

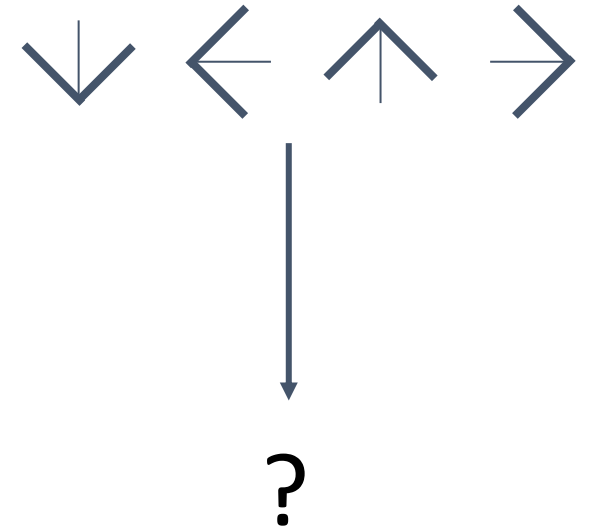
You could try ...



Find edges



Find corners



Machine Learning: Data-Driven Approach

1. Collect a dataset of images and labels
2. Use Machine Learning to train a classifier
3. Evaluate the classifier on new images

Example training set

```
def train(images, labels):  
    # Machine learning!  
    return model
```

```
def predict(model, test_images):  
    # Use model to predict labels  
    return test_labels
```

airplane



automobile



bird



cat



deer



Image Classification Datasets: MNIST



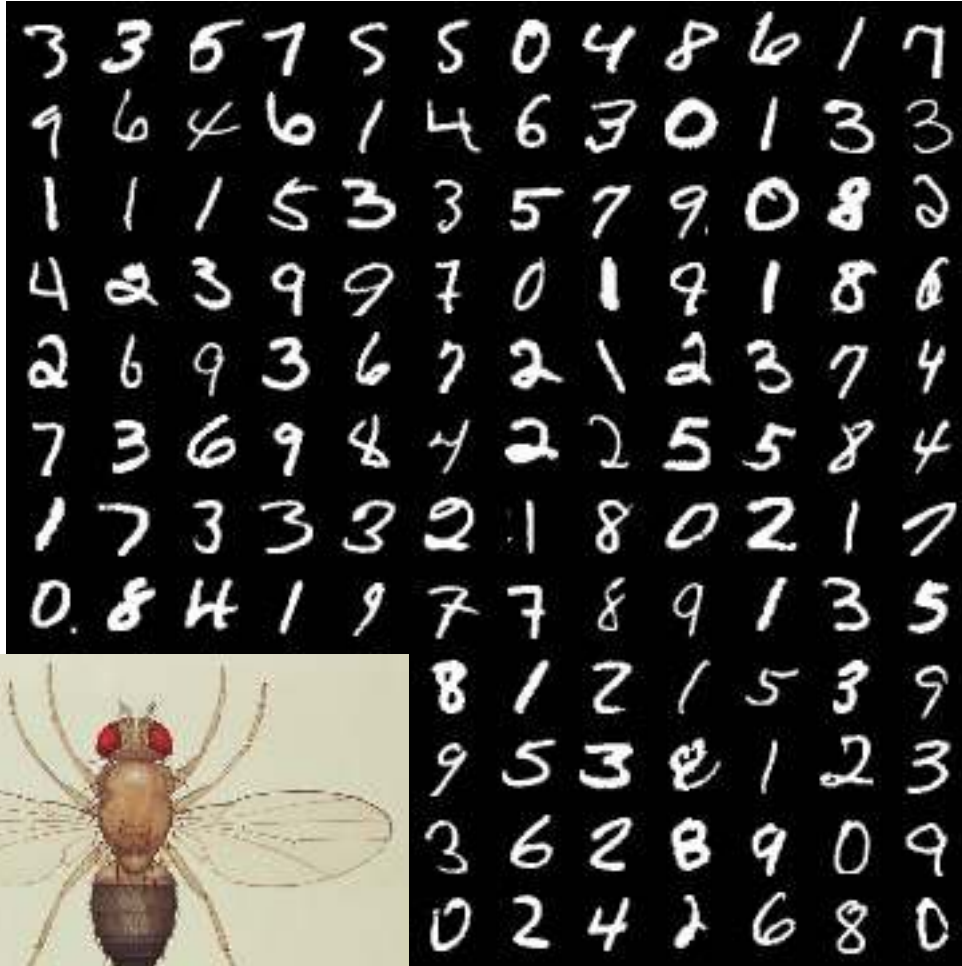
10 classes: Digits 0 to 9

28x28 grayscale images

50k training images

10k test images

Image Classification Datasets: MNIST



10 classes: Digits 0 to 9

28x28 grayscale images

50k training images

10k test images

“Drosophila of computer vision”

Results from MNIST often do not hold on more complex datasets!

Image Classification Datasets: CIFAR10

airplane

automobile

bird

cat

deer

dog

frog

horse

ship

truck



10 classes

50k training images (5k per class)

10k testing images (1k per class)

32x32 RGB images

We will use this dataset for
homework assignments

Image Classification Datasets: CIFAR100



100 classes

50k training images (500 per class)

10k testing images (100 per class)

32x32 RGB images

20 superclasses with 5 classes each:

Aquatic mammals: beaver, dolphin, otter, seal, whale

Trees: Maple, oak, palm, pine, willow

Image Classification Datasets: ImageNet

1000 classes

~1.3M training images (~1.3K per class)

50K validation images (50 per class)

100K test images (100 per class)

Performance metric: **Top 5 accuracy**

Algorithm predicts 5 labels for each image; one of them needs to be right



Deng et al, "ImageNet: A Large-Scale Hierarchical Image Database", CVPR 2009
Russakovsky et al, "ImageNet Large Scale Visual Recognition Challenge", IJCV 2015

Image Classification Datasets: ImageNet

1000 classes

~1.3M training images (~1.3K per class)

50K validation images (50 per class)

100K test images (100 per class)

test labels are secret!



Images have variable size, but often resized to **256x256** for training

There is also a 22k category version of ImageNet, but less commonly used

Deng et al, "ImageNet: A Large-Scale Hierarchical Image Database", CVPR 2009
Russakovsky et al, "ImageNet Large Scale Visual Recognition Challenge", IJCV 2015

Image Classification Datasets: MIT Places



365 classes of different scene types

~8M training images

18.25K val images (50 per class)

328.5K test images (900 per class)

Images have variable size, often
resize to **256x256** for training

Zhou et al, "Places: A 10 million Image Database for Scene Recognition", TPAMI 2017

Classification Datasets: Number of Training Pixels

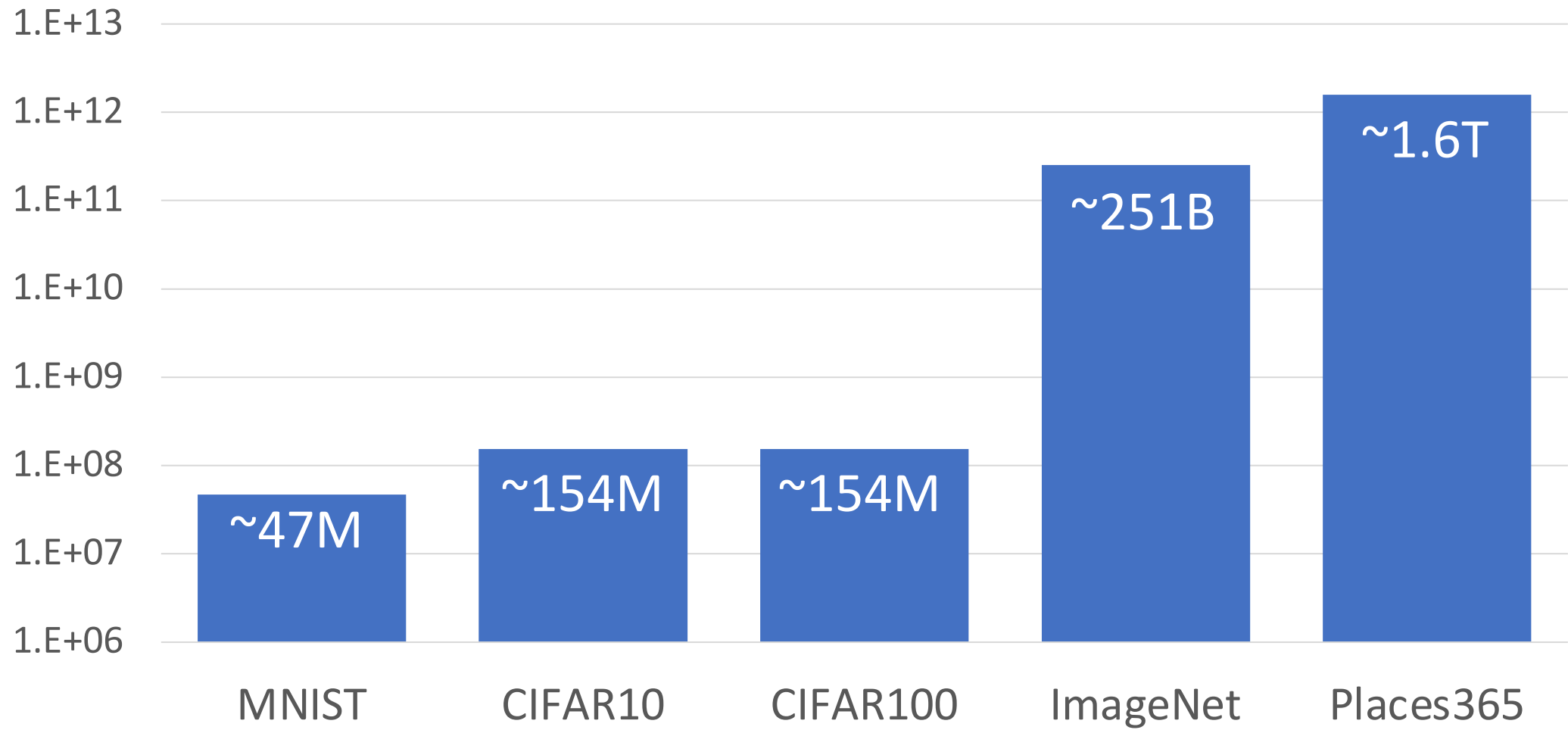


Image Classification Datasets: Omniglot



1623 categories: characters from 50 different alphabets

20 images per category

Meant to test **few shot learning**

Lake et al, "Human-level concept learning through probabilistic program induction", Science, 2015

First classifier: Nearest Neighbor

```
def train(images, labels):  
    # Machine learning!  
    return model
```



Memorize all data
and labels

```
def predict(model, test_images):  
    # Use model to predict labels  
    return test_labels
```



Predict the label of
the most similar
training image

Distance Metric to compare images

L1 distance:

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$

test image

56	32	10	18
90	23	128	133
24	26	178	200
2	0	255	220

training image

10	20	24	17
8	10	89	100
12	16	178	170
4	32	233	112

-

=

pixel-wise absolute value differences

46	12	14	1
82	13	39	33
12	10	0	30
2	32	22	108

add
→ 456

Nearest Neighbor Classifier

```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred
```

Nearest Neighbor Classifier

Memorize training data

```
import numpy as np
```

```
class NearestNeighbor:
```

```
    def __init__(self):
```

```
        pass
```

```
    def train(self, X, y):
```

```
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
```

```
        # the nearest neighbor classifier simply remembers all the training data
```

```
        self.Xtr = X
```

```
        self.ytr = y
```

```
    def predict(self, X):
```

```
        """ X is N x D where each row is an example we wish to predict label for """
```

```
        num_test = X.shape[0]
```

```
        # lets make sure that the output type matches the input type
```

```
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)
```

```
        # loop over all test rows
```

```
        for i in xrange(num_test):
```

```
            # find the nearest training image to the i'th test image
```

```
            # using the L1 distance (sum of absolute value differences)
```

```
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
```

```
            min_index = np.argmin(distances) # get the index with smallest distance
```

```
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example
```

```
        return Ypred
```

Nearest Neighbor Classifier

```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred
```

For each test image:
Find nearest training image
Return label of nearest image


```

import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred

```

Nearest Neighbor Classifier

Q: With N examples,
how fast is training?


```

import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred

```

Nearest Neighbor Classifier

Q: With N examples,
how fast is training?

A: $O(1)$

```

import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred

```

Nearest Neighbor Classifier

Q: With N examples,
how fast is training?

A: $O(1)$

Q: With N examples,
how fast is testing?

```

import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred

```

Nearest Neighbor Classifier

Q: With N examples,
how fast is training?

A: $O(1)$

Q: With N examples,
how fast is testing?

A: $O(N)$


```

import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred

```

Nearest Neighbor Classifier

Q: With N examples,
how fast is training?

A: $O(1)$

Q: With N examples,
how fast is testing?

A: $O(N)$

This is **bad**: We can
afford slow training, but
we need fast testing!


```

import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred

```

Nearest Neighbor Classifier

There are many methods for fast / approximate nearest neighbors; e.g. see

<https://github.com/facebookresearch/faiss>

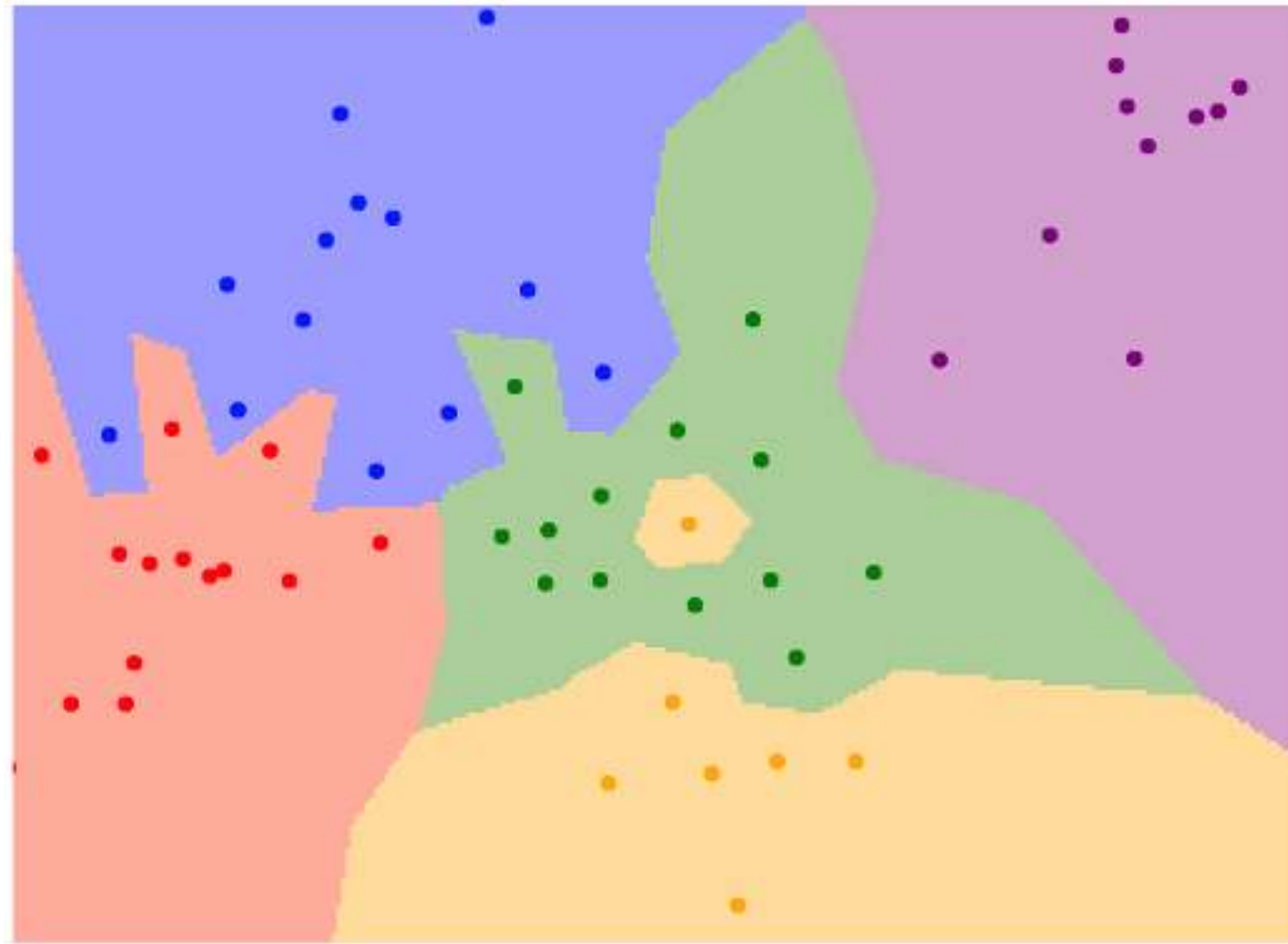
What does this look like?



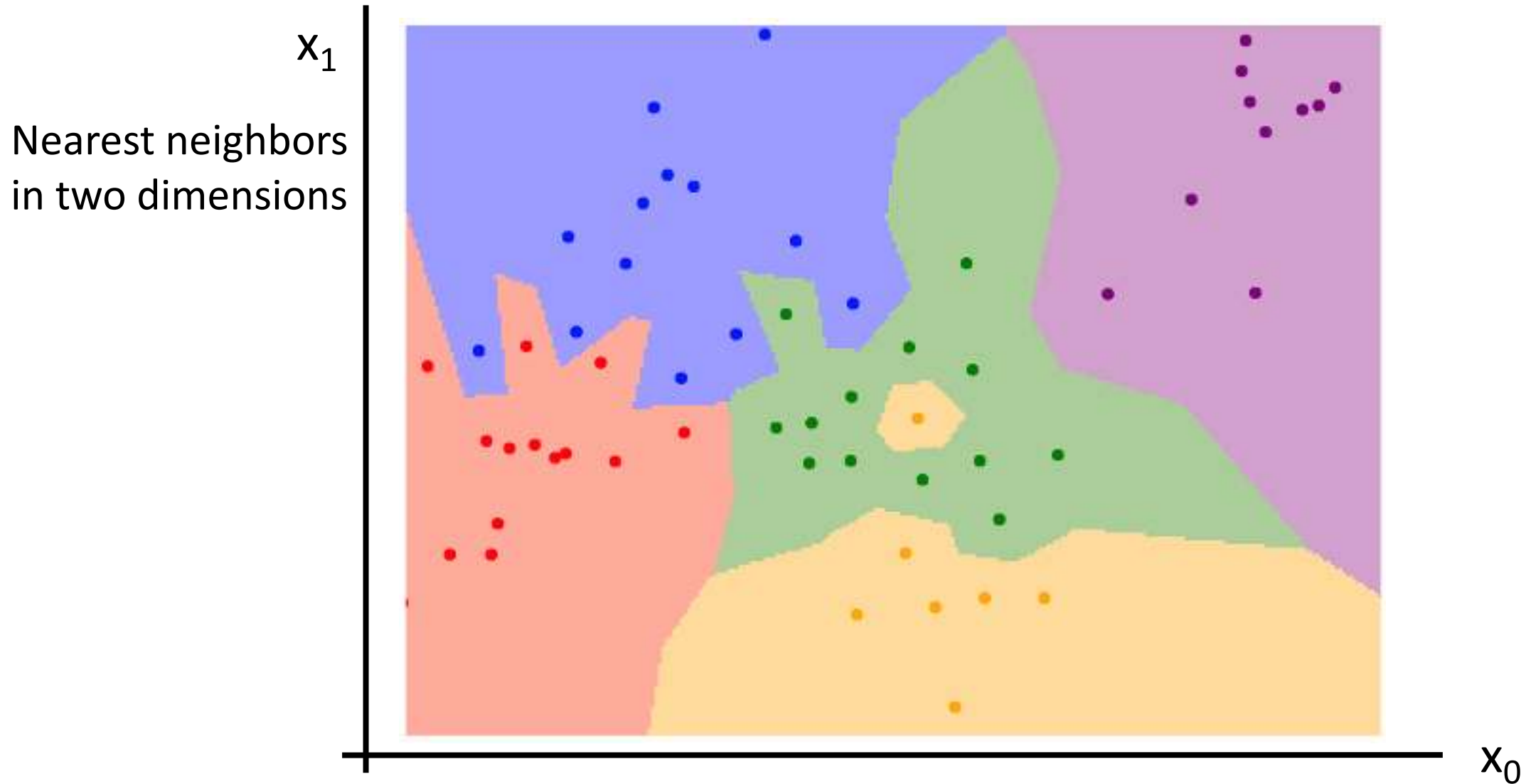
What does this look like?



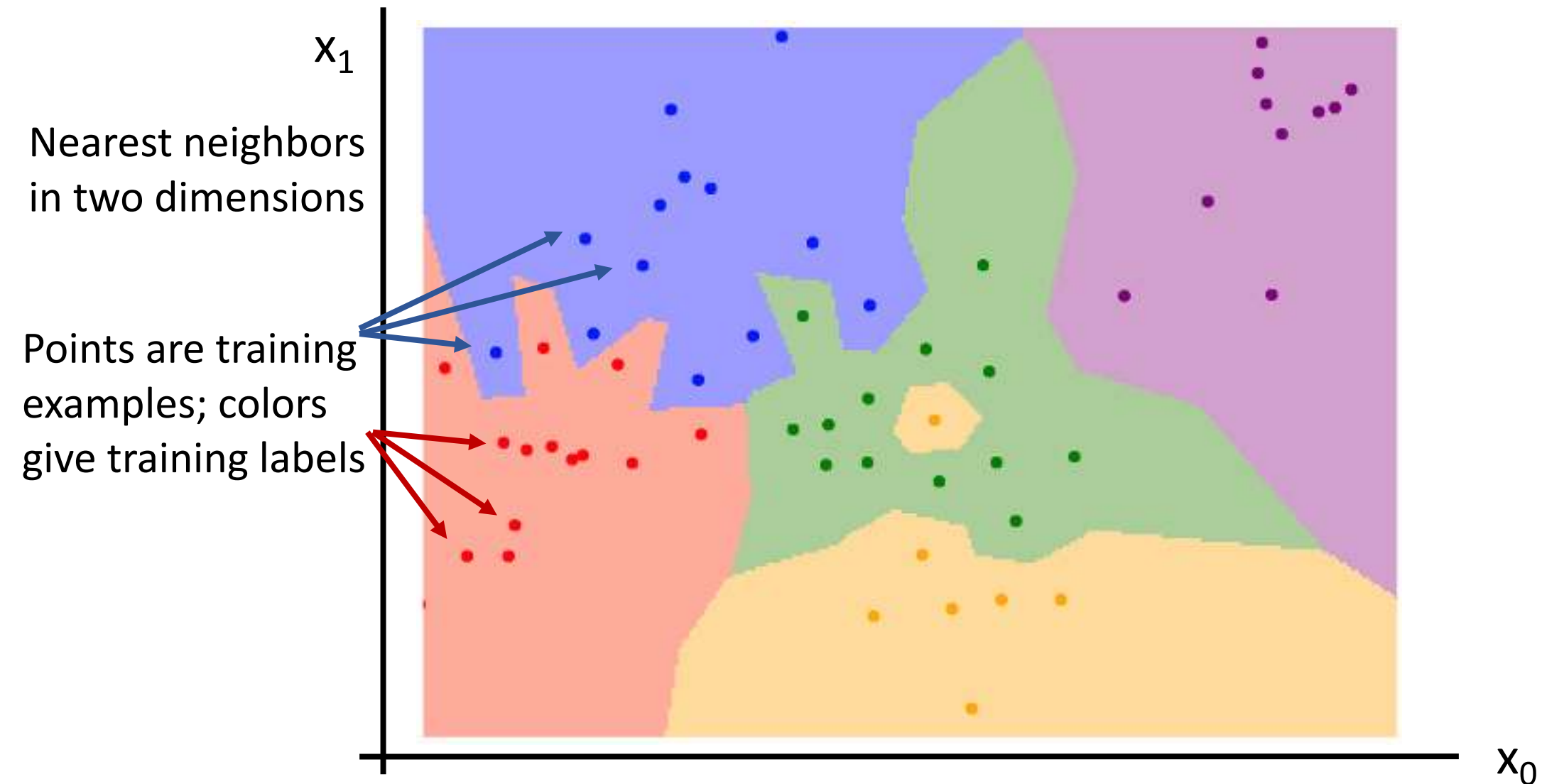
Nearest Neighbor Decision Boundaries



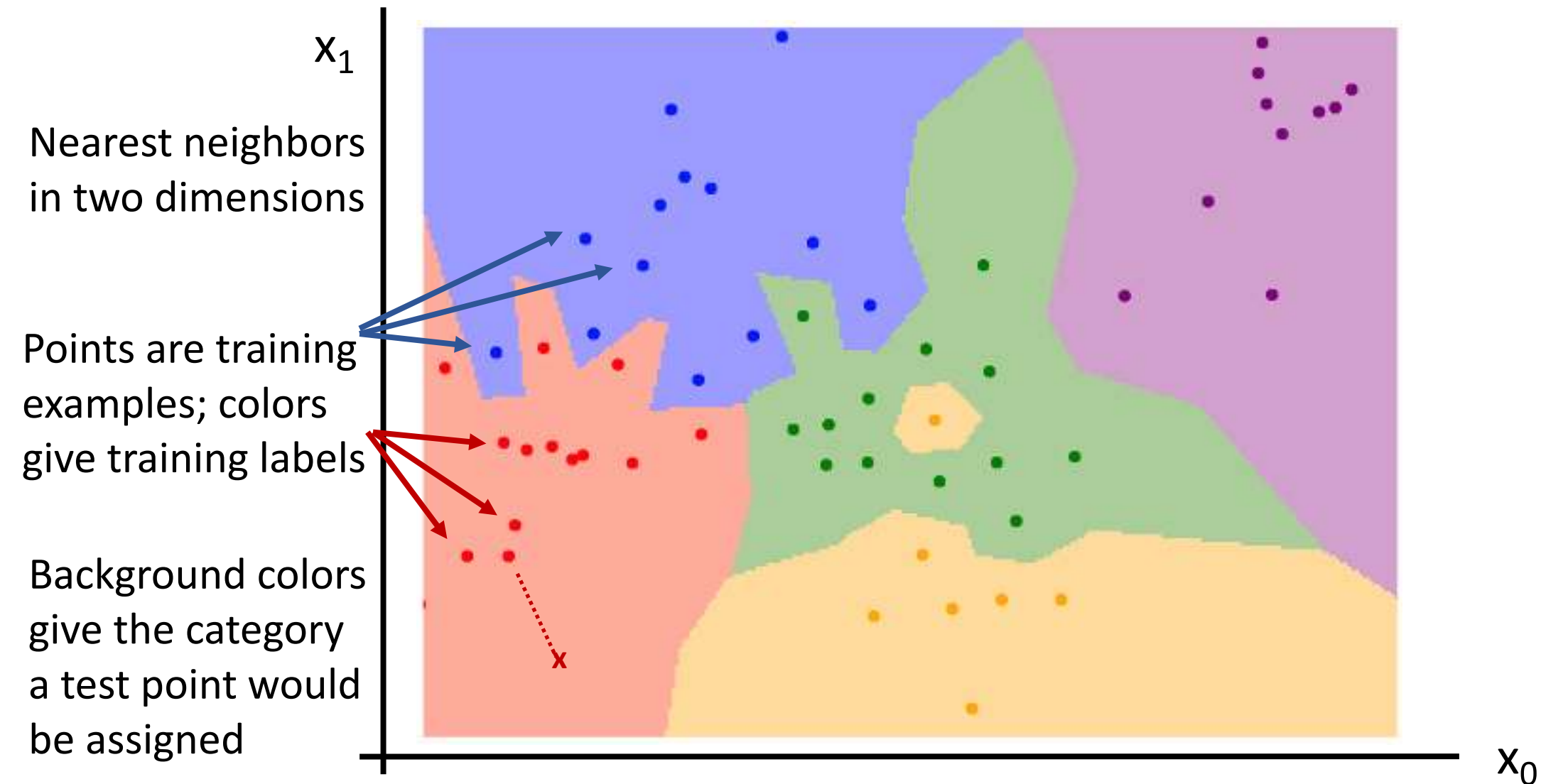
Nearest Neighbor Decision Boundaries



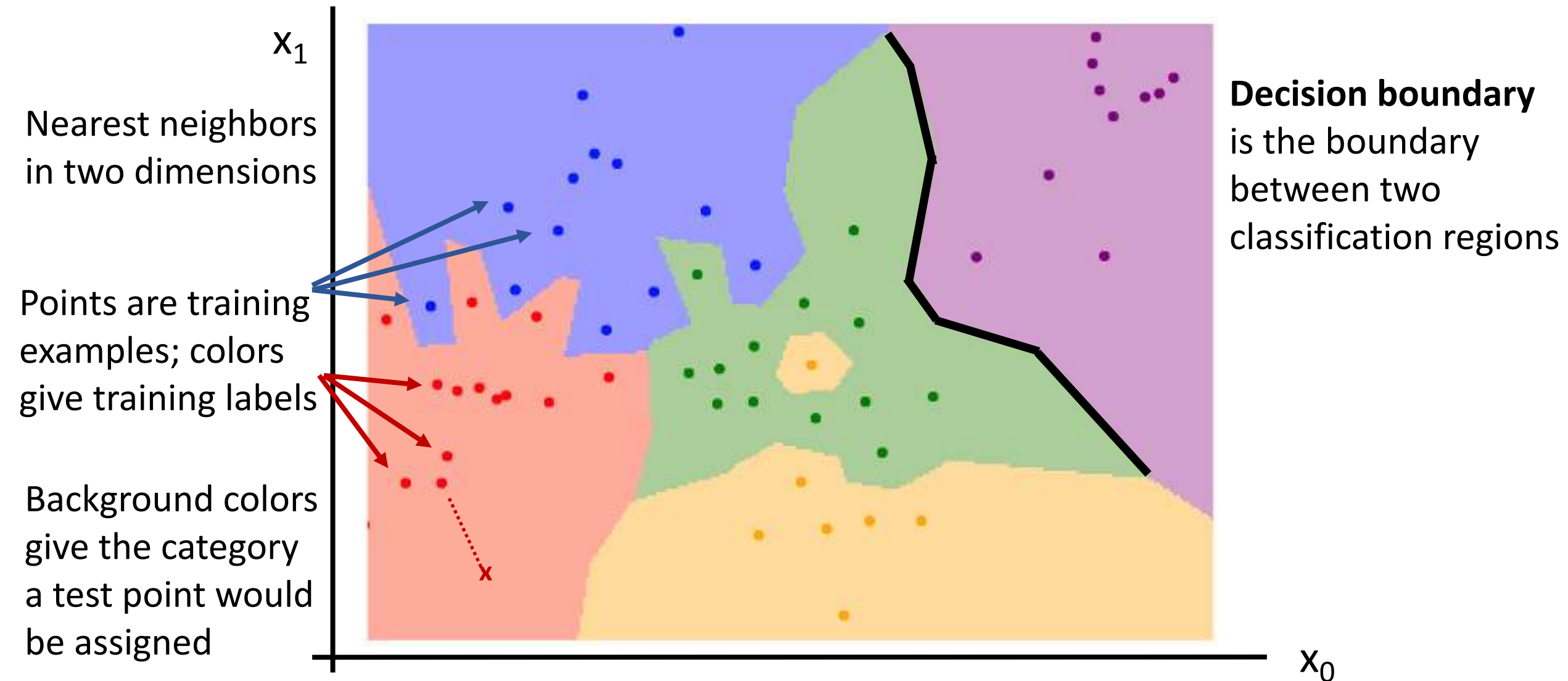
Nearest Neighbor Decision Boundaries



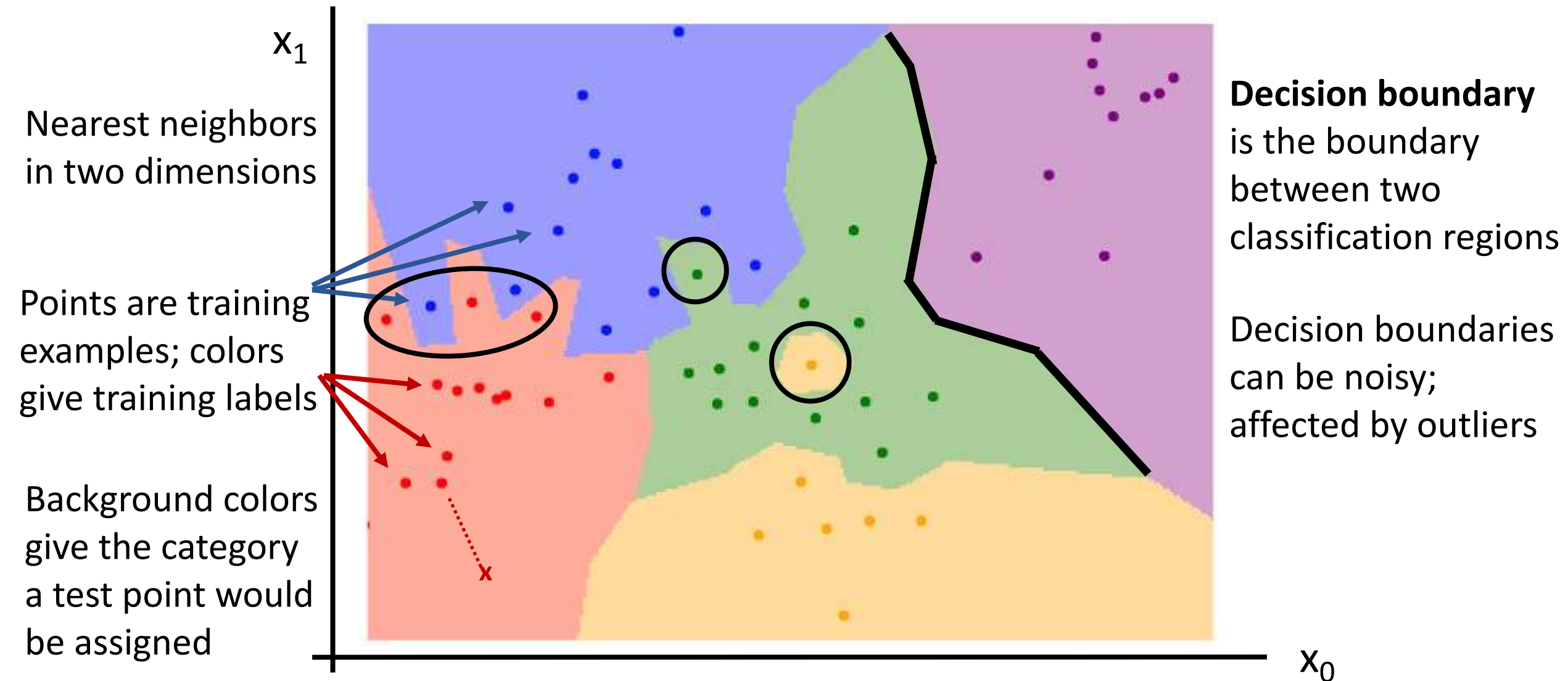
Nearest Neighbor Decision Boundaries



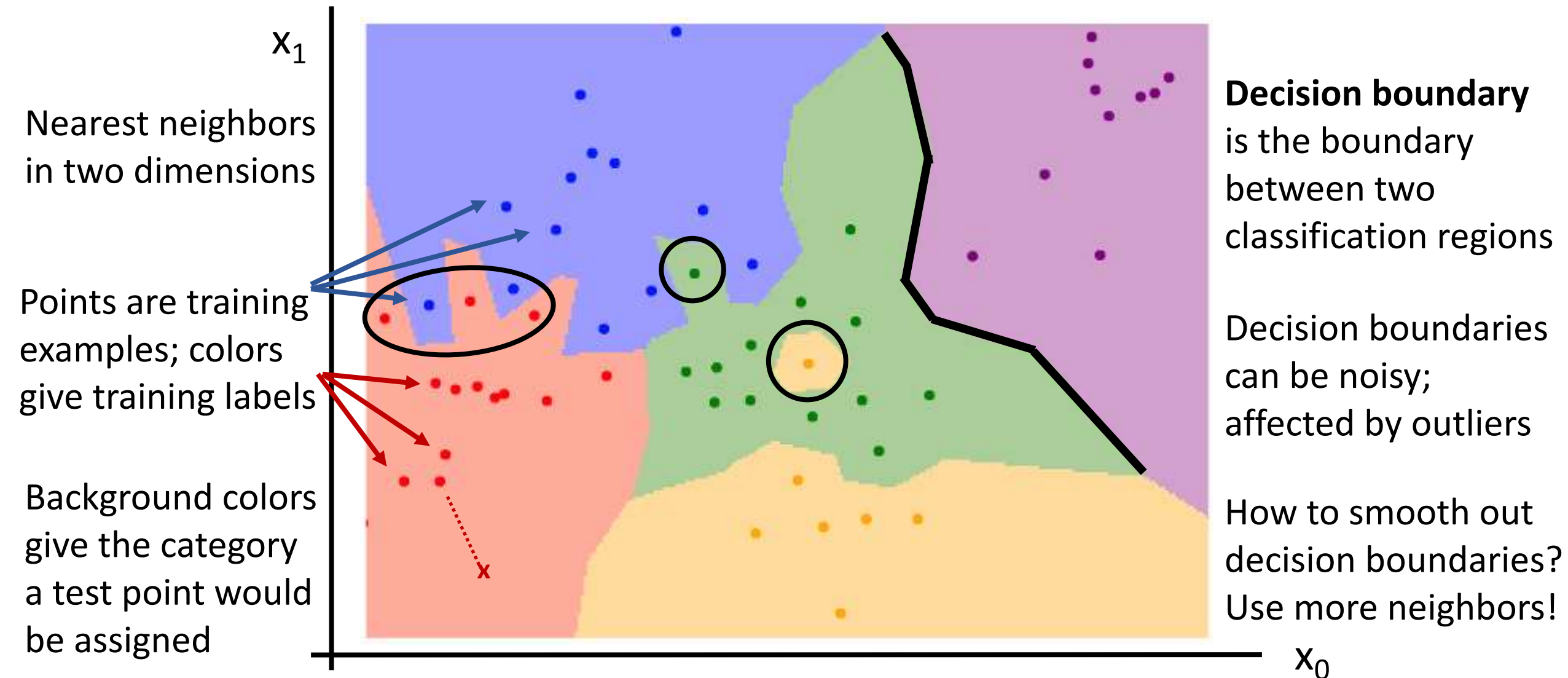
Nearest Neighbor Decision Boundaries



Nearest Neighbor Decision Boundaries



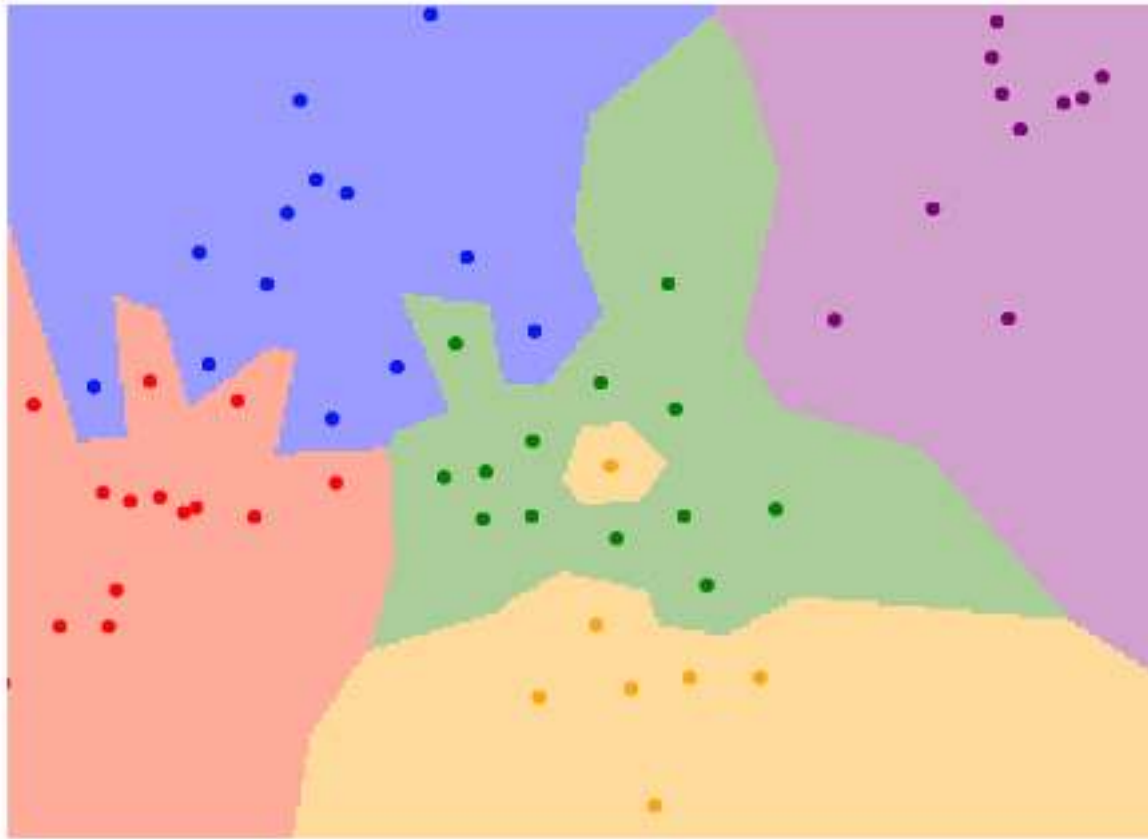
Nearest Neighbor Decision Boundaries



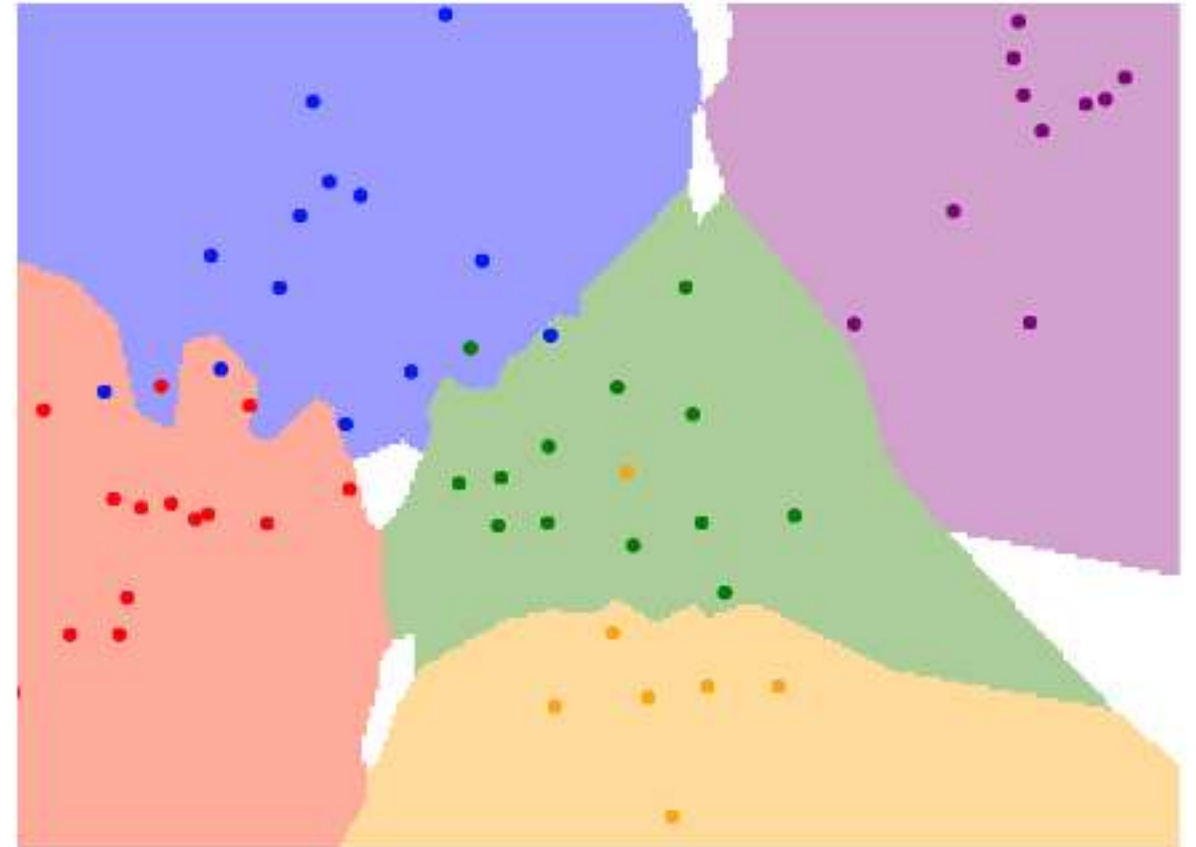
K-Nearest Neighbors

Instead of copying label from nearest neighbor, take **majority vote** from K closest points

$K = 1$



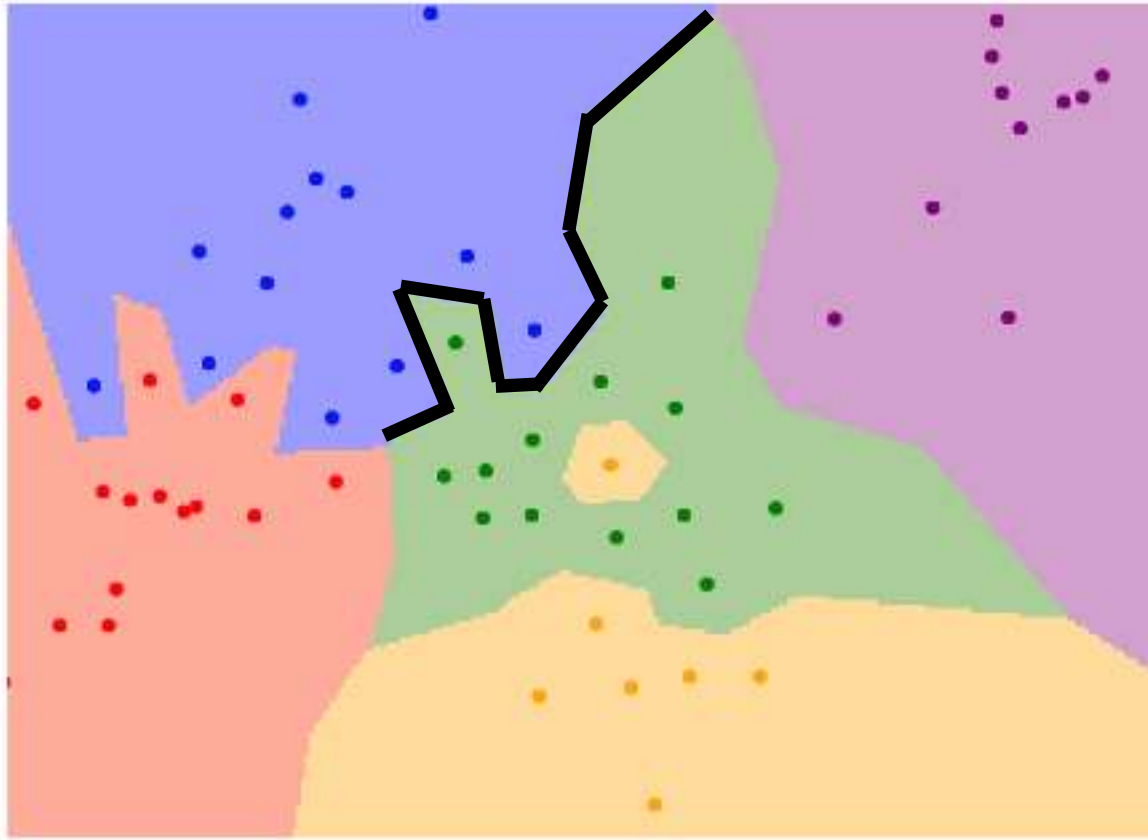
$K = 3$



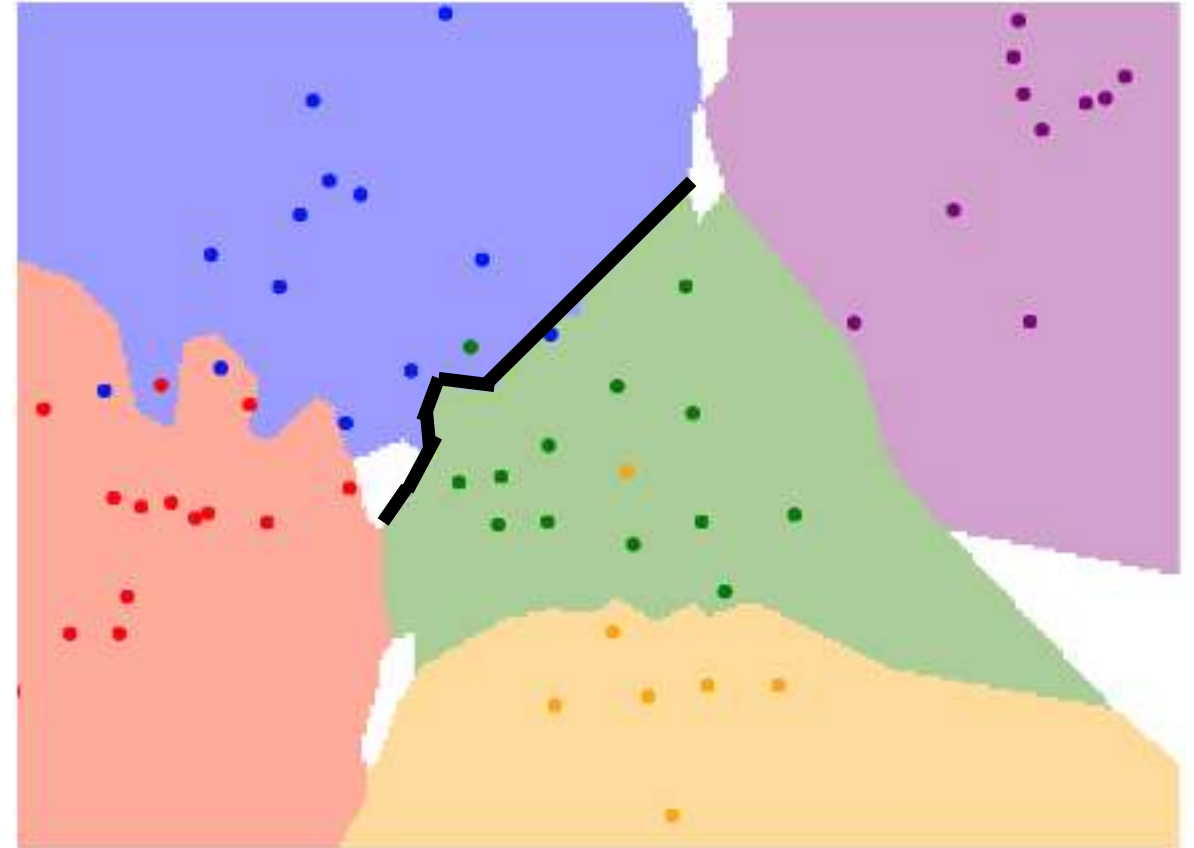
K-Nearest Neighbors

Using more neighbors helps smooth out rough decision boundaries

$K = 1$



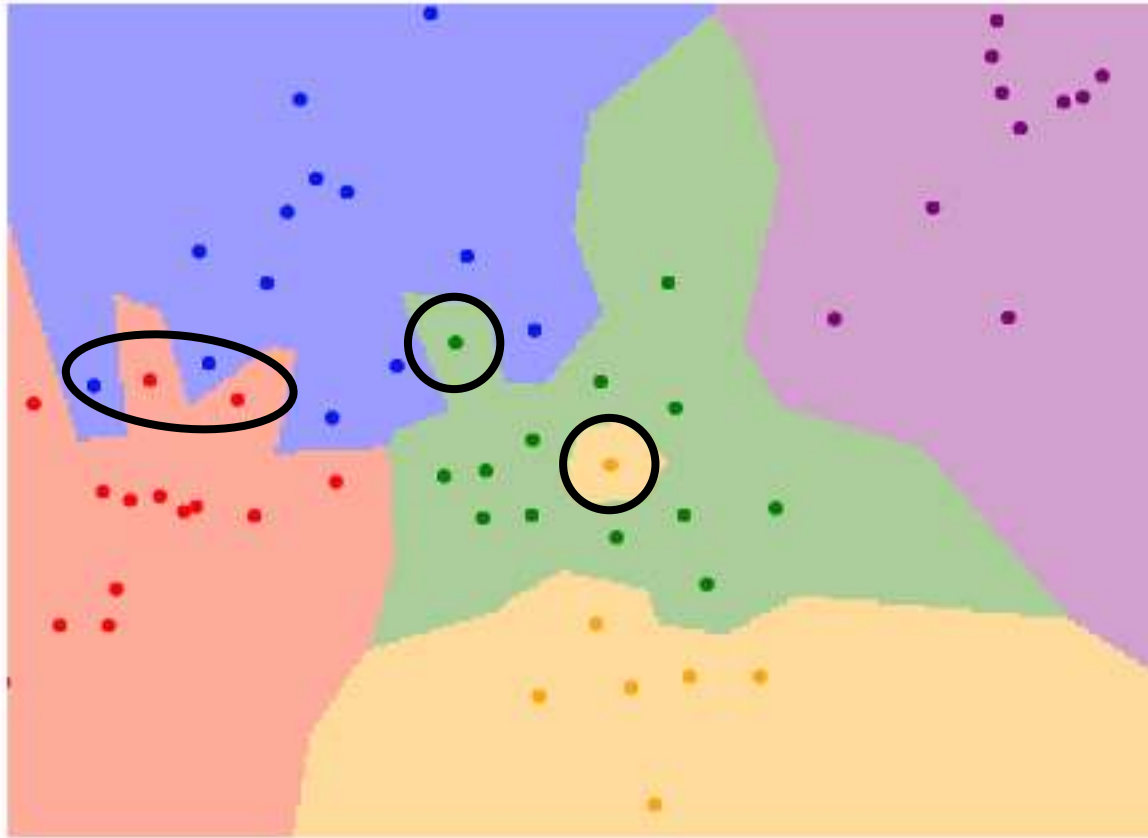
$K = 3$



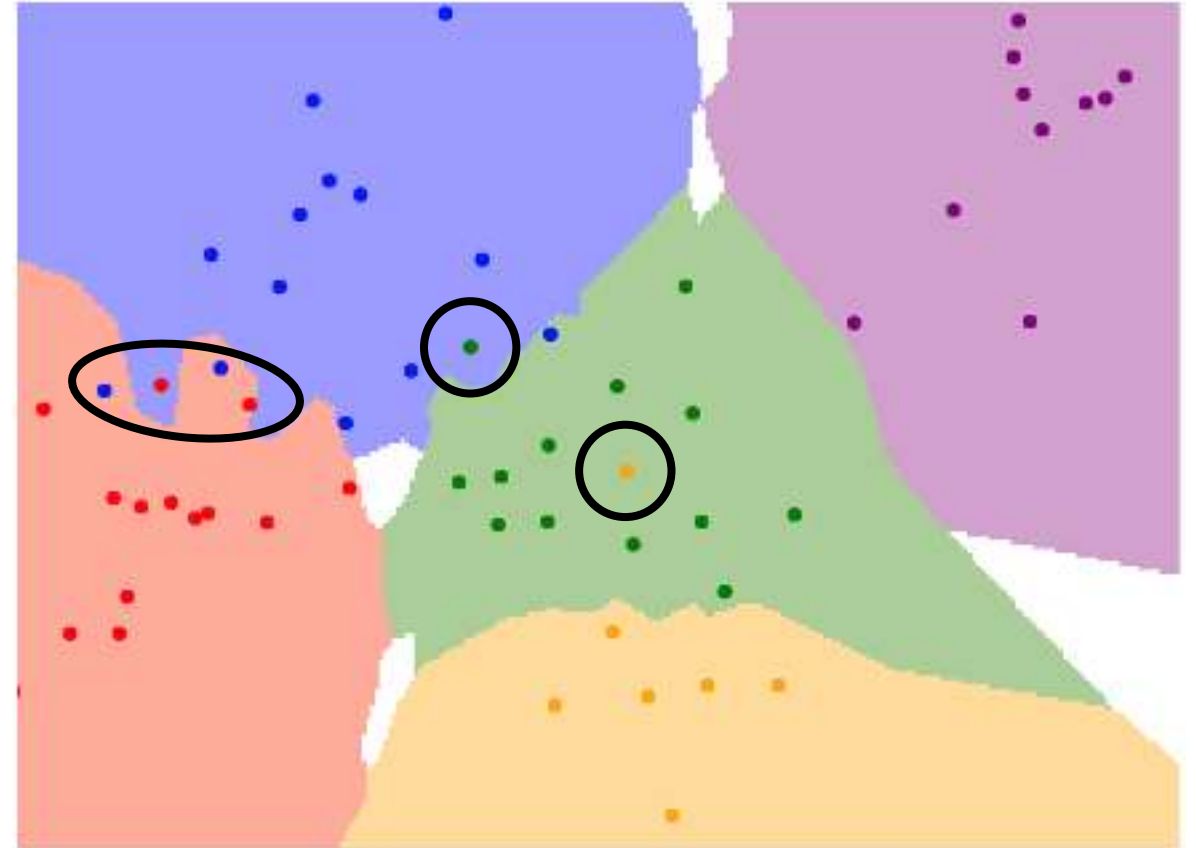
K-Nearest Neighbors

Using more neighbors helps
reduce the effect of outliers

$K = 1$



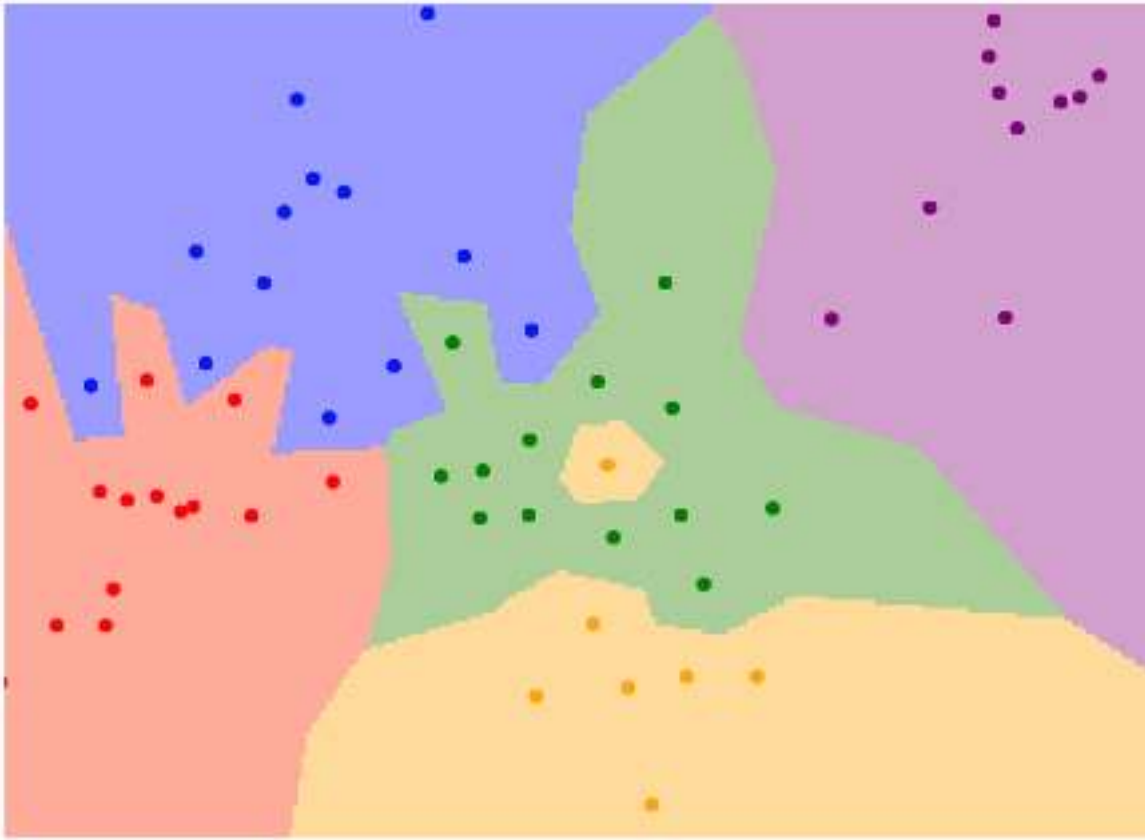
$K = 3$



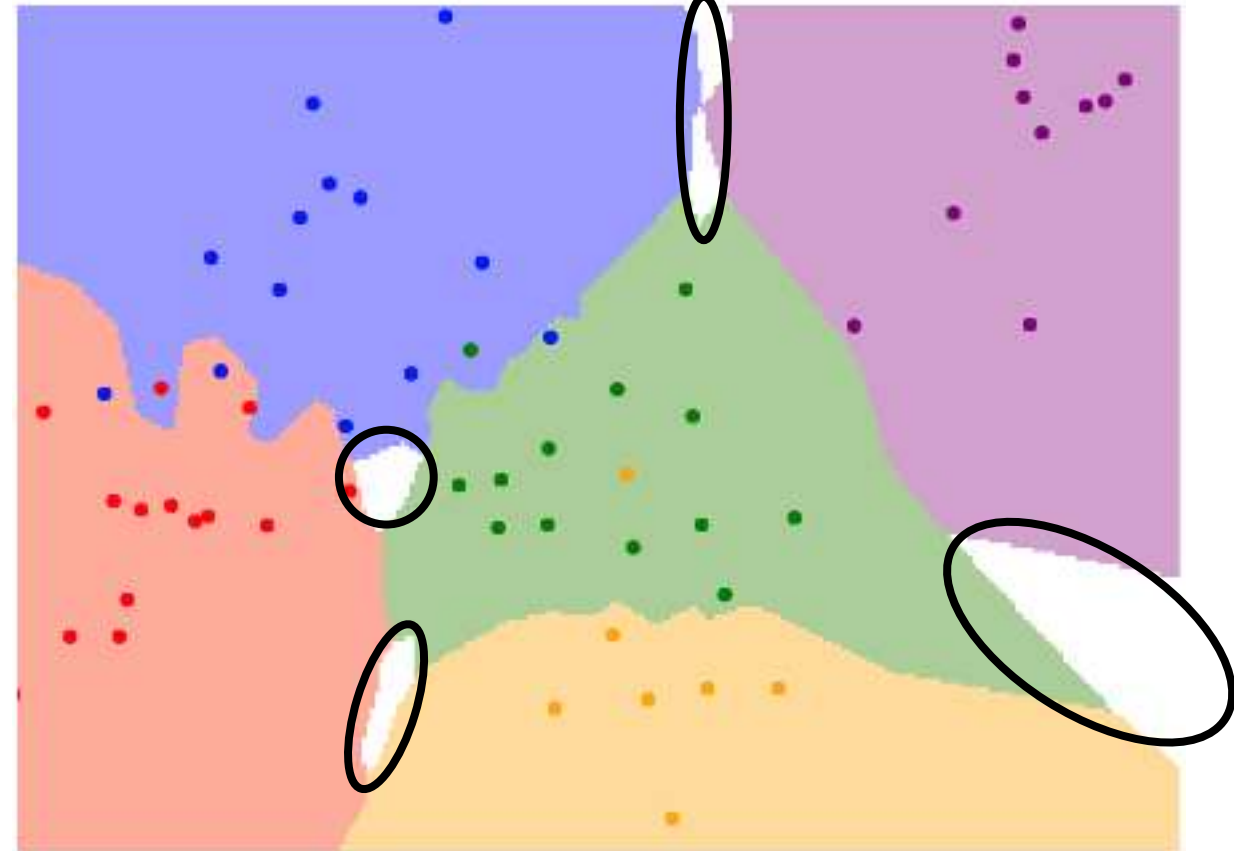
K-Nearest Neighbors

When $K > 1$ there can be ties between classes.
Need to break somehow!

$K = 1$



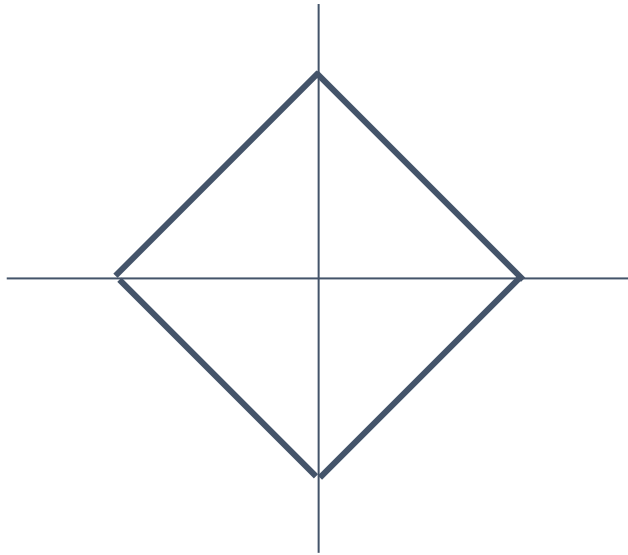
$K = 3$



K-Nearest Neighbors: Distance Metric

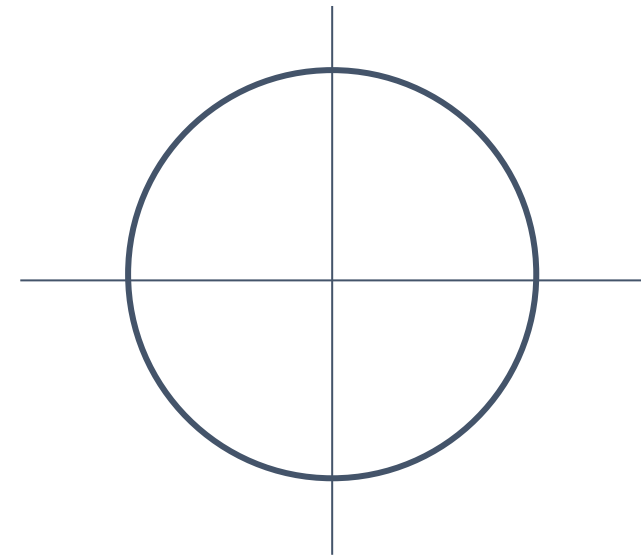
L1 (Manhattan) distance

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$



L2 (Euclidean) distance

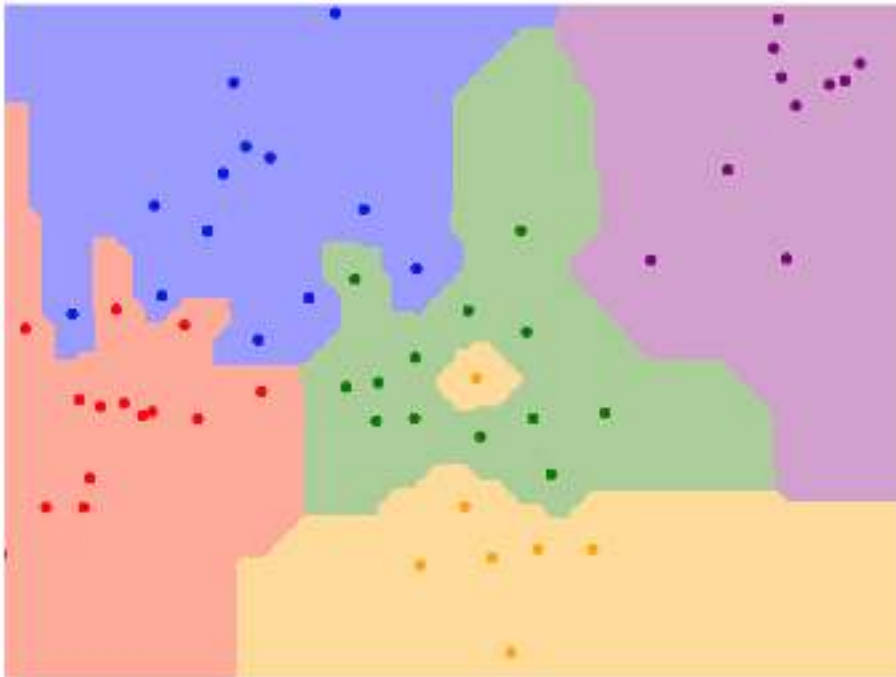
$$d_2(I_1, I_2) = \left(\sum_p (I_1^p - I_2^p)^2 \right)^{\frac{1}{2}}$$



K-Nearest Neighbors: Distance Metric

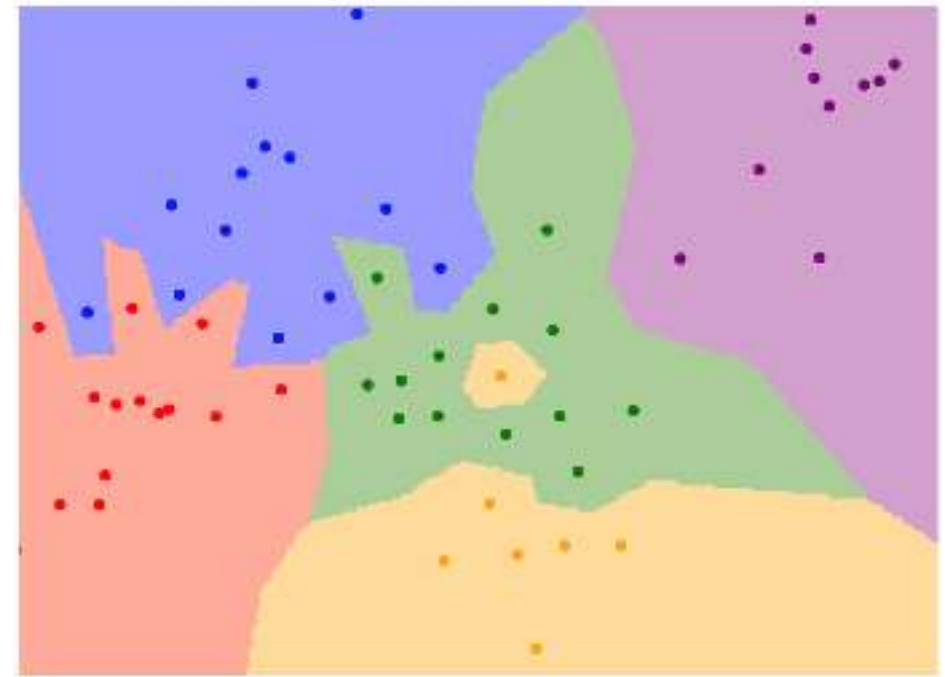
L1 (Manhattan) distance

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$



L2 (Euclidean) distance

$$d_2(I_1, I_2) = \left(\sum_p (I_1^p - I_2^p)^2 \right)^{\frac{1}{2}}$$



K = 1

K-Nearest Neighbors: Distance Metric

With the right choice of distance metric, we can apply K-Nearest Neighbor to any type of data!

K-Nearest Neighbors: Distance Metric

With the right choice of distance metric, we can apply K-Nearest Neighbor to any type of data!

Example:
Compare
research
papers using
tf-idf similarity

Mesh R-CNN

Georgia Gkioxari, Jitendra Malik, Justin Johnson

6/6/2019 cs.CV

1808.02739v1 pdf
show similar discuss



Rapid advances in 2D perception have led to systems that accurately detect objects in real-world images. However, these systems make predictions in 2D, ignoring the 3D structure of the world. Concurrently, advances in 3D shape prediction have mostly focused on synthetic benchmarks and isolated objects. We unify advances in these two areas. We propose a system that detects objects in real-world images and produces a triangle mesh giving the full 3D shape of each detected object. Our system, called Mesh R-CNN, augments Mask R-CNN with a mesh prediction branch that outputs meshes with varying topological structure by first predicting coarse voxel representations which are converted to meshes and refined with a graph convolution network operating over the mesh's vertices and edges. We validate our mesh prediction branch on ShapeNet, where we outperform prior work on single-image shape prediction. We then deploy our full Mesh R-CNN system on Pix3D, where we jointly detect objects and predict their 3D shapes.

<http://www.arxiv-sanity.com/search?q=mesh+r-cnn>

K-Nearest Neighbors: Distance Metric

More similar papers:

Image-based 3D Object Reconstruction: State-of-the-Art and Trends in the Deep Learning Era

Xian Peng Han, Harid Lagr, Mohamed Benamoun

5/10/2019 (v1: 6/10/2019) cs.CV cs.CG cs.GR cs.LG

1005.10648v2 pdf

[show similar](#) [discuss](#)



3D reconstruction is a longstanding ill-posed problem, which has been explored for decades by the computer vision, computer graphics, and machine learning communities. Since 2016, image-based 3D reconstruction using convolutional neural networks (CNNs) has attracted increasing interest and demonstrated an impressive performance. Given this new era of rapid evolution, this article provides a comprehensive survey of the recent developments in this field. We focus on the works which use deep learning techniques to estimate the 3D shape of generic objects either from a single or multiple RGB images. We organize the literature based on the shape representations, the network architectures, and the training methods they use. While this survey is intended for methods which reconstruct generic objects, we also review some of the recent works which focus on specific object classes such as human body shapes and faces. We provide an analysis and comparison of the performance of some key papers, summarize some of the open problems in this field, and discuss promising directions for future research.

Pix2Mesh: Generating 3D Mesh Models from Single RGB Images

Yansong Wang, Yinda Zhang, Zhuwen Li, Yifan Fu, Wei Liu, Yu-Ceng Yang

5/3/2019 (v1: 4/5/2019) cs.CV

1906.01594v2 pdf

[show similar](#) [discuss](#)



We propose an end-to-end deep learning architecture that encodes a 3D shape in triangular mesh from a single color image. Limited by the nature of shape and texture, previous methods usually represent a 3D shape in volume or point cloud, and then manually convert them to the more ready-to-use mesh model. Unlike the existing methods, our network represents 3D mesh in a graph-based convolutional neural network and produces correct geometry by progressively deforming an ellipsoid. Leveraging perceptual features extracted from the input image, we adopt a coarse-to-fine strategy to make the whole deformation procedure stable, and define various of mesh-related losses to capture properties of different levels to guarantee visually appealing and physically accurate 3D geometry. Extensive experiments show that our method not only qualitatively produces mesh model with better details, but also achieves higher 3D shape estimation accuracy compared to the state-of-the-art.

Pix2Mesh++: Multi-View 3D Mesh Generation via Deformation

Chen Wen, Yinda Zhang, Zhuwen Li, Yifan Fu

10/10/2019 (v1: 5/27/2019) cs.CV

Accepted by ICCV 2019

1005.01407v2 pdf

[show similar](#) [discuss](#)



We study the problem of shape generation in 3D mesh representation from a few color images with known camera poses. While many previous works learn to initialize the shape directly from pixels, we resort to further improving the shape quality by leveraging cross-view information with a graph convolutional network instead of building a direct mapping function from images to 3D shape. Our model learns to predict series of deformation to improve a coarse shape iteratively. Inspired by traditional multiple view geometry methods, our network samples nearby area around the initial mesh's vertex location and uses their optimal deformation using perceptual feature statistics built from multiple input images. Extra view information is shown that our model produces accurate 3D shapes that are not only visually plausible from the input perspectives, but also well aligned to arbitrary viewpoints. With the help of physically driven architecture, our model also exhibits generalization capability across different semantic categories, number of input images, and quality of mesh initialization.

GEOMetrics: Exploiting Geometric Structure for Graph-Encoded Objects

Edward J. Smith, Scott Fujimori, Antonio Romero, David Meger

10/1/2019 cs.CV

10 pages

1801.11457v1 pdf

[show similar](#) [discuss](#)



Mesh models are a promising approach for encoding the structure of 3D objects. Current mesh reconstruction systems predict uniformly distributed vertex locations of a predetermined graph through a series of graph convolutions, leading to compromises with respect to performance or resolution. In this paper, we argue that the graph representation of geometric objects allows for additional structure, which should be leveraged for enhanced reconstruction. Thus, we propose a system which properly harnesses from the advantages of the geometric structure of graph-encoded objects by introducing (1) a graph convolutional update preserving vertex information, (2) an adaptive splitting heuristic allowing detail to emerge, and (3) a training objective operating both on the local surfaces defined by vertices as well as the global structure defined by the mesh. Our proposed method is evaluated on the task of 3D shape reconstruction from images with the ShapeNet dataset, where we demonstrate state-of-the-art performance, both visually and numerically, while having the smallest memory requirements by generating adaptive meshes.

<http://www.arxiv-sanity.com/1906.02739v1>

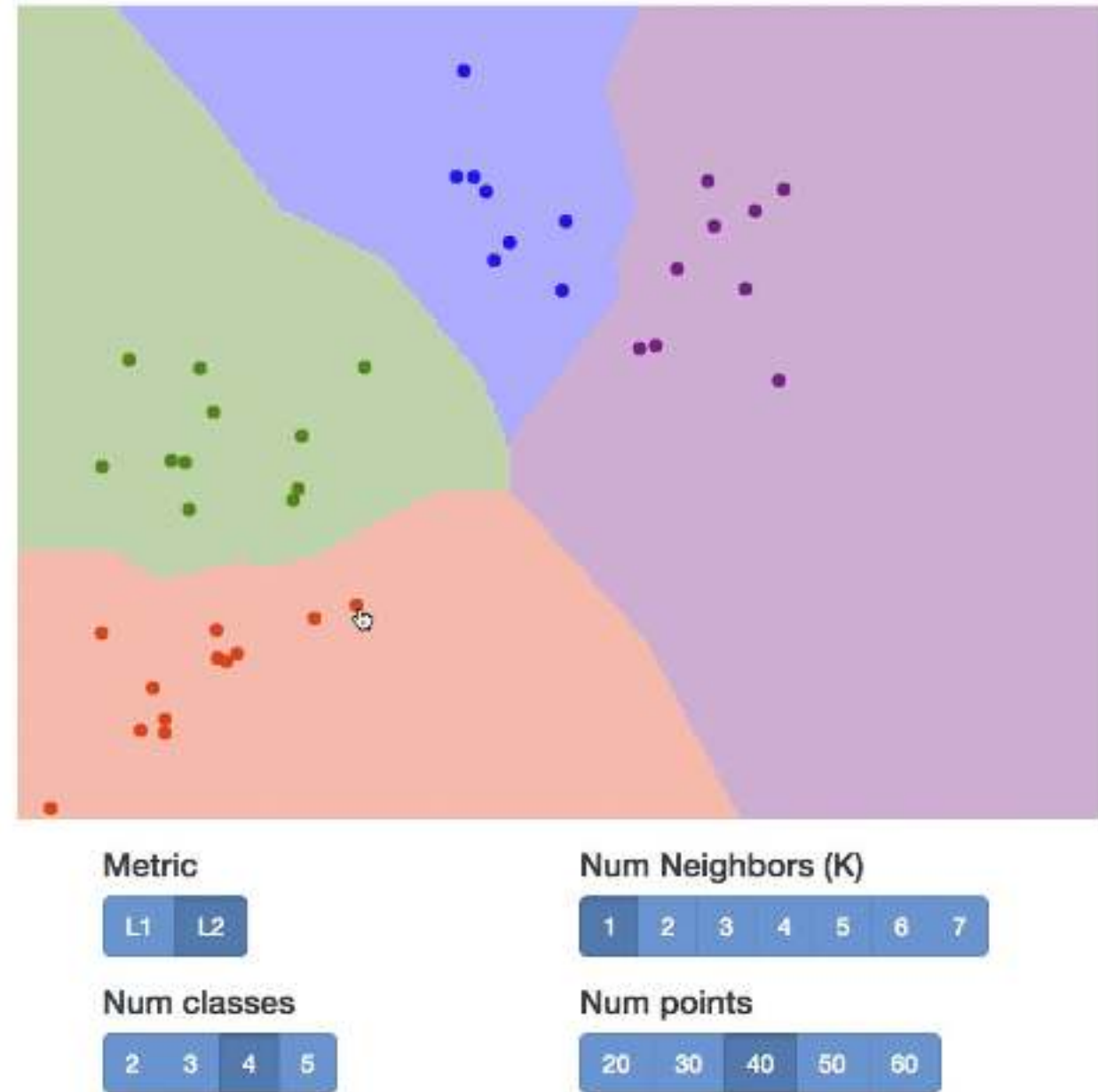
K-Nearest Neighbors: Web Demo

Interactively move points around
and see decision boundaries change

Play with L1 vs L2 metrics

Play with changing number of
training points, value of K

<http://vision.stanford.edu/teaching/cs231n-demos/knn/>



Hyperparameters

What is the best value of **K** to use?

What is the best **distance metric** to use?

These are examples of **hyperparameters**: choices about our learning algorithm that we don't learn from the training data; instead we set them at the start of the learning process

Hyperparameters

What is the best value of **K** to use?

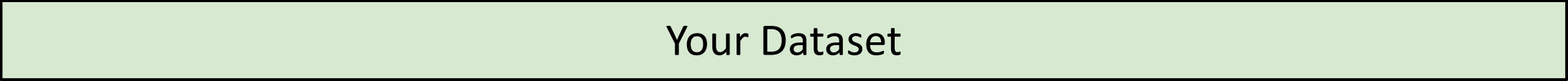
What is the best **distance metric** to use?

These are examples of **hyperparameters**: choices about our learning algorithm that we don't learn from the training data; instead we set them at the start of the learning process

Very problem-dependent. In general need to try them all and see what works best for our data / task.

Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the data

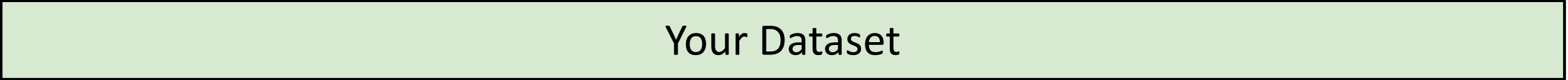


Your Dataset

Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the data

BAD: $K = 1$ always works perfectly on training data



Your Dataset

Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the data

BAD: $K = 1$ always works perfectly on training data



Your Dataset

Idea #2: Split data into **train** and **test**, choose hyperparameters that work best on test data



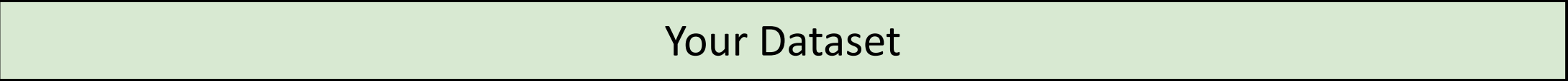
train

test

Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the data

BAD: $K = 1$ always works perfectly on training data



Your Dataset

Idea #2: Split data into **train** and **test**, choose hyperparameters that work best on test data

BAD: No idea how algorithm will perform on new data



train

test

Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the data

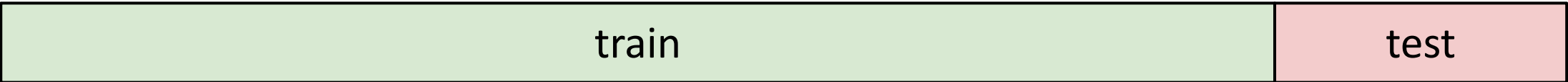
BAD: $K = 1$ always works perfectly on training data



Your Dataset

Idea #2: Split data into **train** and **test**, choose hyperparameters that work best on test data

BAD: No idea how algorithm will perform on new data

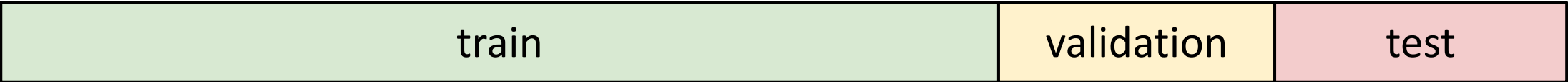


train

test

Idea #3: Split data into **train**, **val**, and **test**; choose hyperparameters on val and evaluate on test

Better!



train

validation

test

Setting Hyperparameters

Your Dataset

Idea #4: Cross-Validation: Split data into **folds**, try each fold as validation and average the results

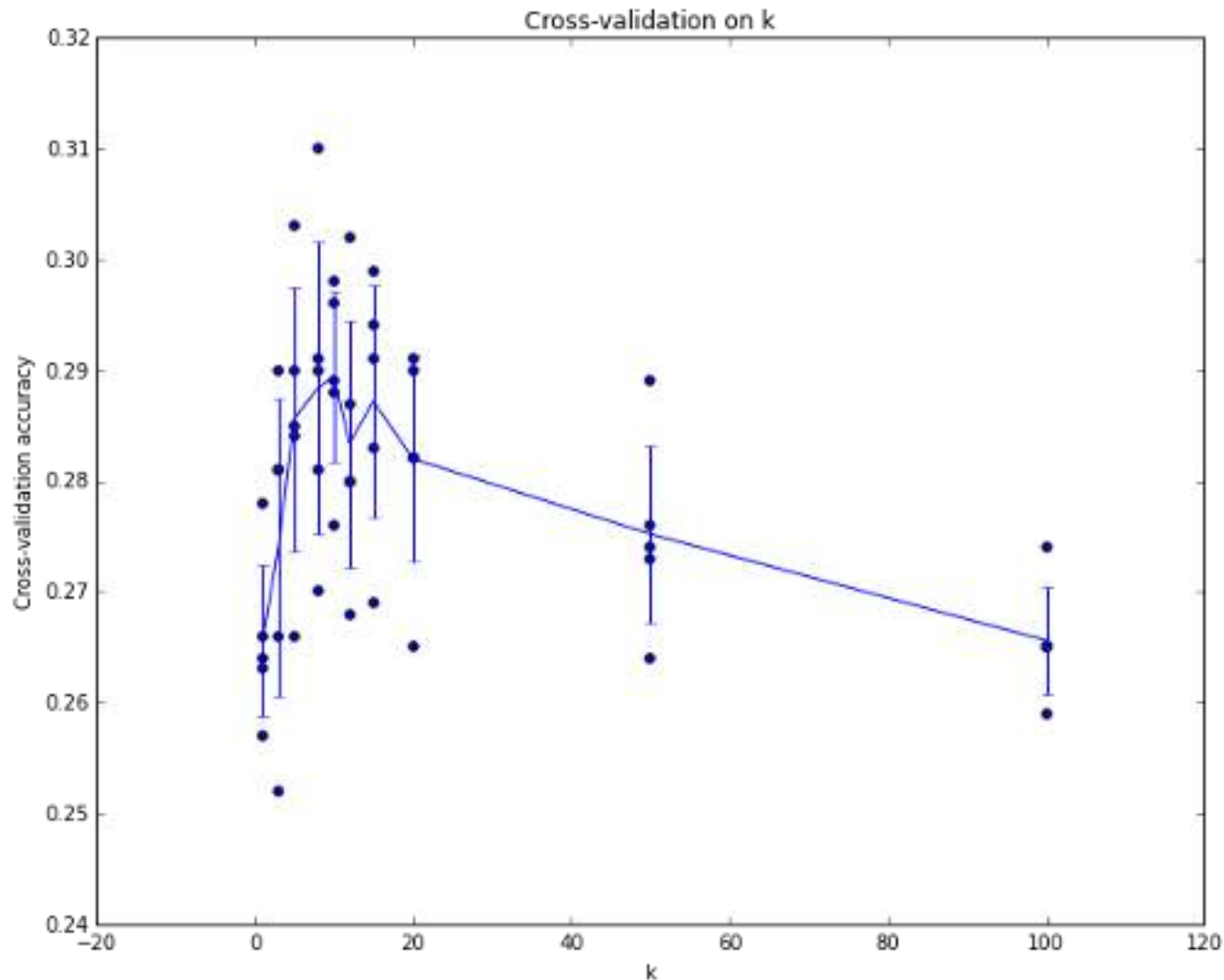
fold 1	fold 2	fold 3	fold 4	fold 5	test
--------	--------	--------	--------	--------	------

fold 1	fold 2	fold 3	fold 4	fold 5	test
--------	--------	--------	--------	--------	------

fold 1	fold 2	fold 3	fold 4	fold 5	test
--------	--------	--------	--------	--------	------

Useful for small datasets, but (unfortunately) not used too frequently in deep learning

Setting Hyperparameters



Example of 5-fold cross-validation for the value of k .

Each point: single outcome.

The line goes through the mean, bars indicated standard deviation

(Seems that $k \sim 7$ works best for this data)

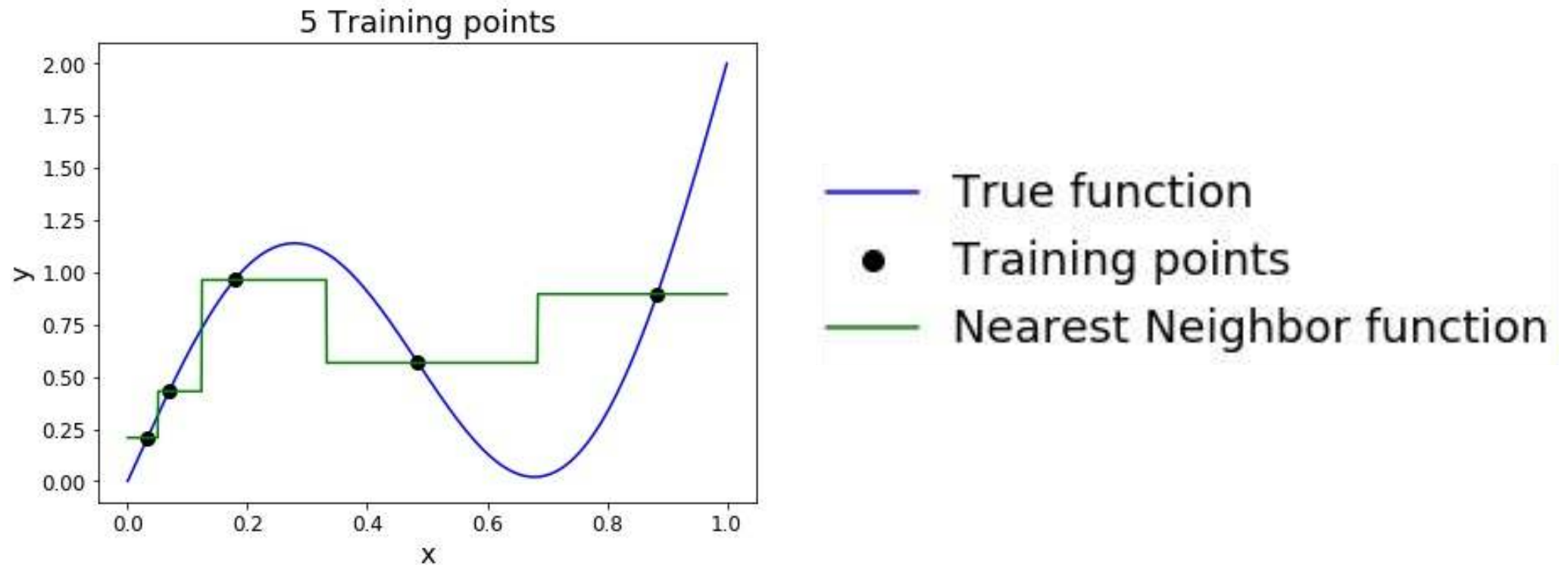
K-Nearest Neighbor: Universal Approximation

As the number of training samples goes to infinity, nearest neighbor can represent any^(*) function!

(*) Subject to many technical conditions. Only continuous functions on a compact domain; need to make assumptions about spacing of training points; etc.

K-Nearest Neighbor: Universal Approximation

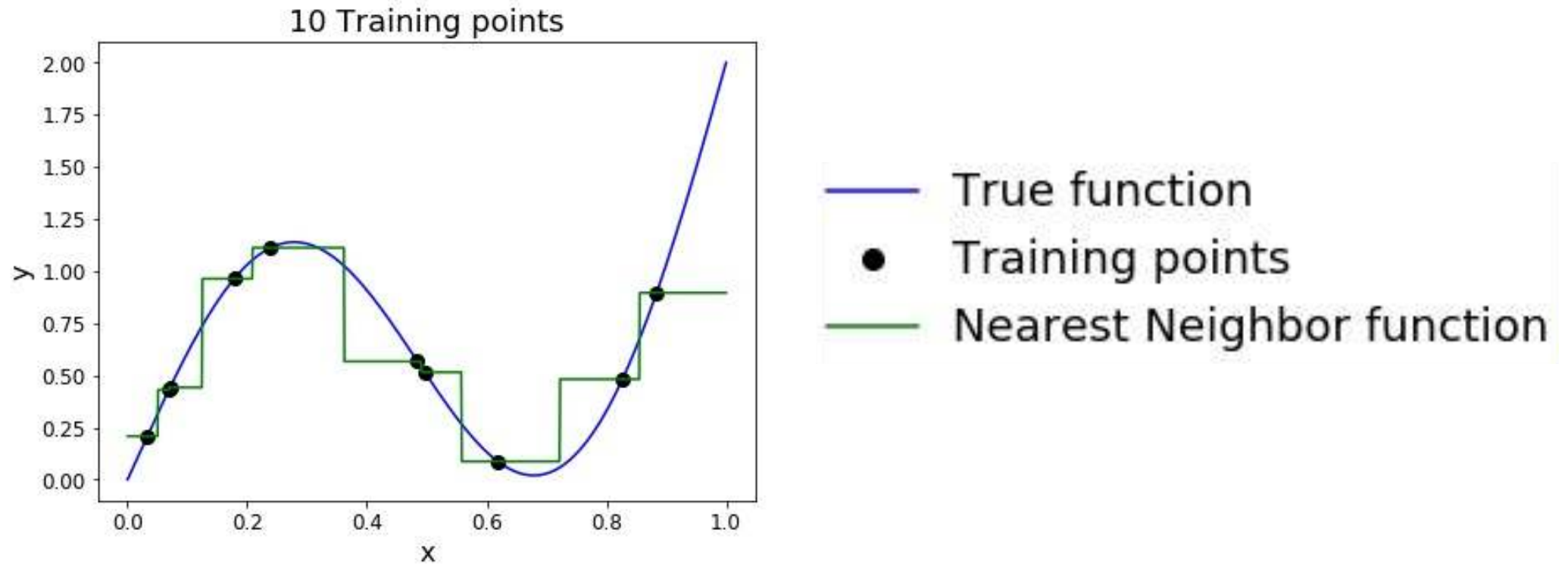
As the number of training samples goes to infinity, nearest neighbor can represent any^(*) function!



(*) Subject to many technical conditions. Only continuous functions on a compact domain; need to make assumptions about spacing of training points; etc.

K-Nearest Neighbor: Universal Approximation

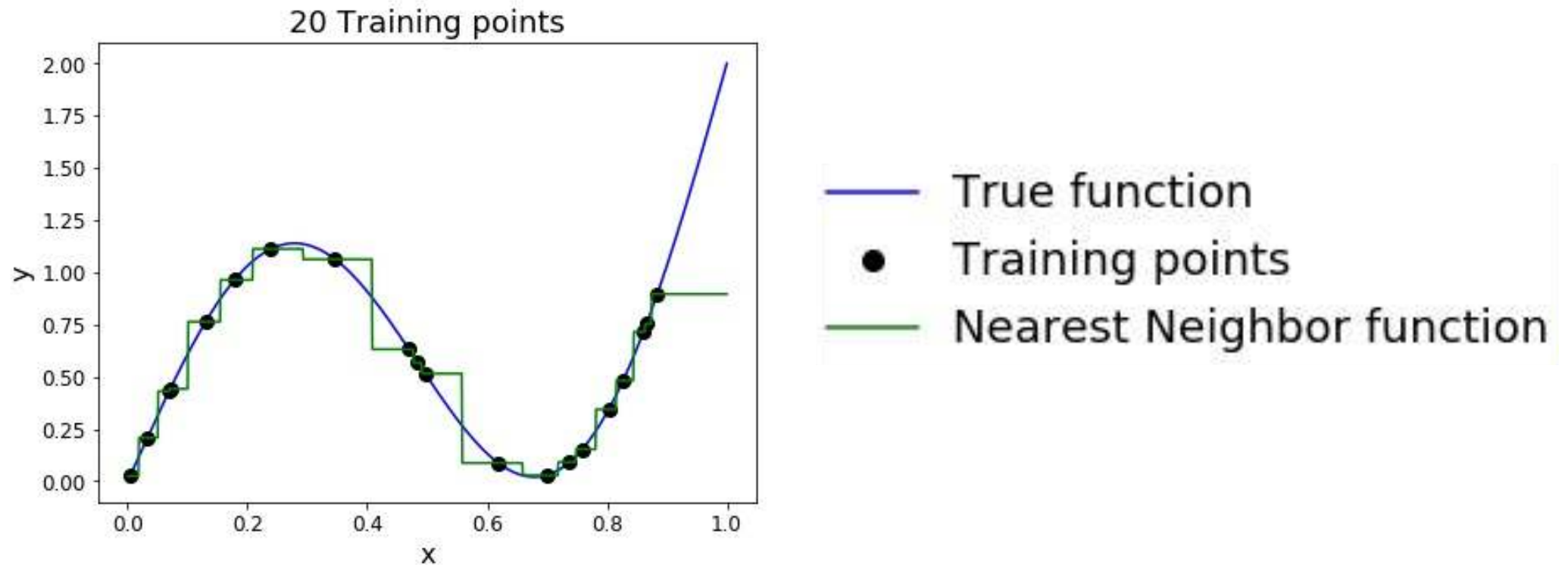
As the number of training samples goes to infinity, nearest neighbor can represent any^(*) function!



(*) Subject to many technical conditions. Only continuous functions on a compact domain; need to make assumptions about spacing of training points; etc.

K-Nearest Neighbor: Universal Approximation

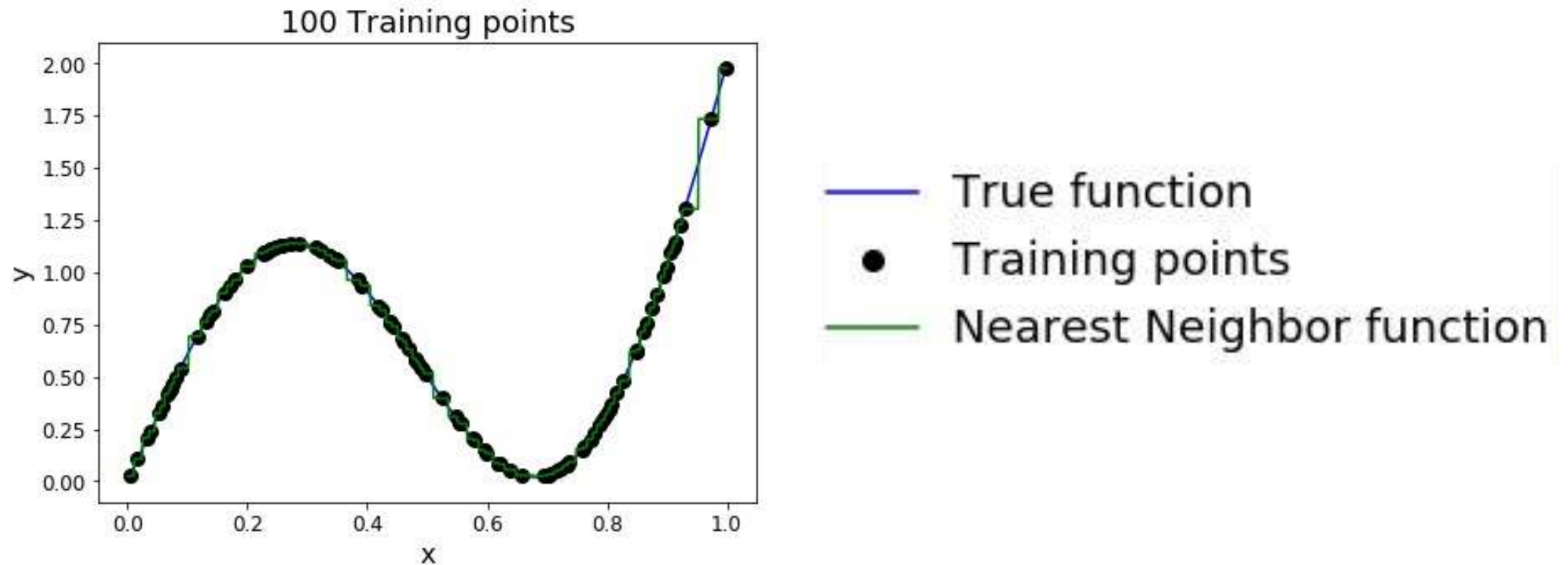
As the number of training samples goes to infinity, nearest neighbor can represent any^(*) function!



(*) Subject to many technical conditions. Only continuous functions on a compact domain; need to make assumptions about spacing of training points; etc.

K-Nearest Neighbor: Universal Approximation

As the number of training samples goes to infinity, nearest neighbor can represent any^(*) function!



(*) Subject to many technical conditions. Only continuous functions on a compact domain; need to make assumptions about spacing of training points; etc.

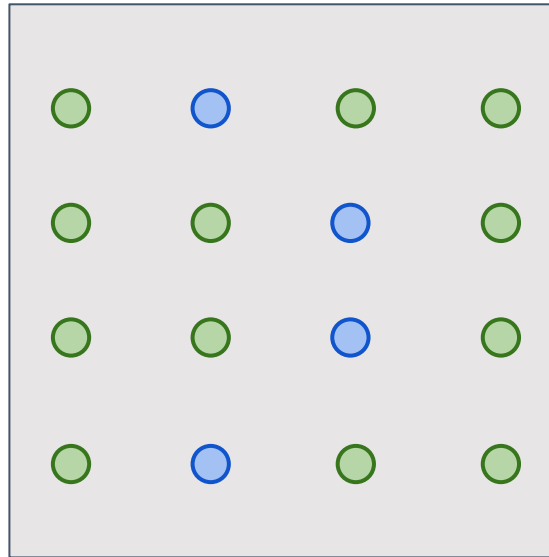
Problem: Curse of Dimensionality

Curse of dimensionality: For uniform coverage of space, number of training points needed grows exponentially with dimension

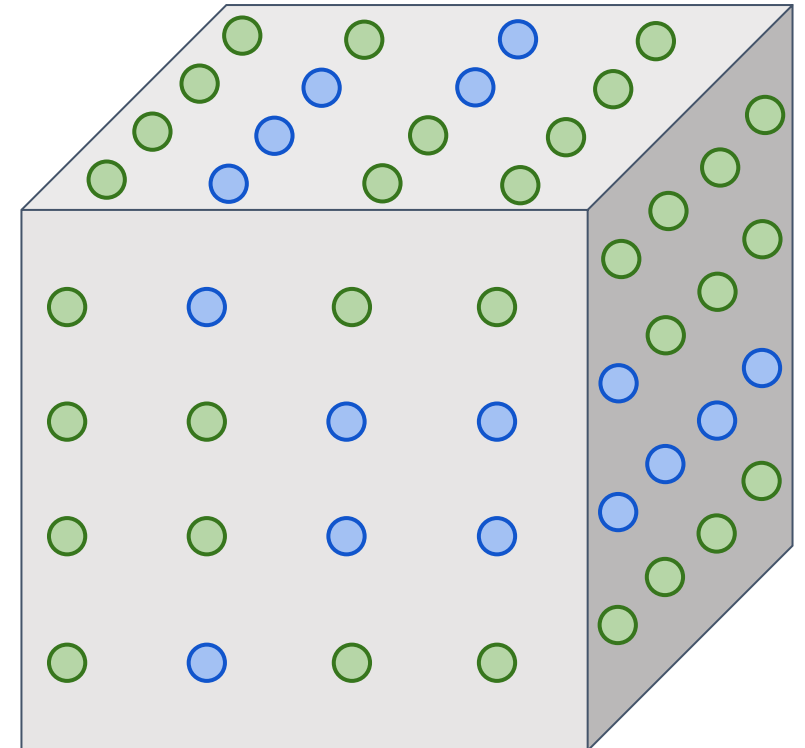
Dimensions = 1
Points = 4



Dimensions = 2
Points = 4^2



Dimensions = 3
Points = 4^3



Problem: Curse of Dimensionality

Curse of dimensionality: For uniform coverage of space, number of training points needed grows exponentially with dimension

Number of possible
32x32 binary images:

$$2^{32 \times 32} \approx 10^{308}$$

Problem: Curse of Dimensionality

Curse of dimensionality: For uniform coverage of space, number of training points needed grows exponentially with dimension

Number of possible
32x32 binary images:

$$2^{32 \times 32} \approx 10^{308}$$

Number of elementary particles
in the visible universe: [\(source\)](#)

$$\approx 10^{97}$$

K-Nearest Neighbor on raw pixels is seldom used

- Very slow at test time
- Distance metrics on pixels are not informative

Original



Boxed



Shifted



Tinted



(all 3 images have same L2 distance to the one on the left)

[Original image](#) is
[CC0 public domain](#)

Nearest Neighbor with ConvNet features works well!



Devlin et al, "Exploring Nearest Neighbor Approaches for Image Captioning", 2015

Nearest Neighbor with ConvNet features works well!

Example: Image Captioning with Nearest Neighbor



A bedroom with a bed and a couch.



A cat sitting in a bathroom sink.



A train is stopped at a train station.



A wooden bench in front of a building.

Devlin et al, "Exploring Nearest Neighbor Approaches for Image Captioning", 2015

Summary

In **Image classification** we start with a **training set** of images and labels, and must predict labels on the **test set**

Image classification is challenging due to the semantic gap: we need invariance to occlusion, deformation, lighting, intraclass variation, etc

Image classification is a **building block** for other vision tasks

The **K-Nearest Neighbors** classifier predicts labels based on nearest training examples

Distance metric and K are **hyperparameters**

Choose hyperparameters using the **validation set**; only run on the test set once at the very end!

Next time: Linear Classifiers

